Mining the Design Space of Multi-Agent Workflows

Xavier Cadet[α,*] Junyan Cheng[α,*] Peter Chin[α] Kyle Richardson[β]
[α]Dartmouth College [β]Allen Institute for AI
[*]Equal contribution.
{xavier.fjf.cadet,jc.th}@dartmouth.edu kyler@allenai.org

Website junyan.ch/apex-paper **Code** github.com/xcadet/apex

***Abstract*—Designing large language model (LLM) multi-agent systems (MAS) still relies on trial and error and informal "folk wisdom." Manual studies test only a few communication structures or topologies (e.g., sequential, parallel, shared-memory, or debate), while automated optimizers discard search trajectories after selecting a task-specific workflow. We present a two-phase pipeline for mining reproducible design principles from workflow code and execution traces. We repurpose AFlow's MCTS optimizer across six composable operators and 26 deterministically verified Enigmata tasks, retaining all intermediate rounds to build a corpus of 28,568 executable workflows. A sandboxed AutoResearch loop then mines orchestration code and prompt templates under an L_0-penalized validation objective and tests candidate principles on held-out searches. Across 300 feature-mining trials, we identify effects that persist on held-out searches across three levels of workflow design: workflows beginning with independent solvers and one-shot aggregation (Star-first) perform worse ($\Delta = -0.045$); persistent shared documents (SharedMemory) outperform debate ($\Delta = +0.135$); and prompt specificity and placeholder-based context routing yield gains comparable to communication-structure selection ($\Delta = +0.118$). More broadly, our pipeline and results show the feasibility of turning discarded search trajectories into a statistical testbed for discovering and validating multi-agent workflow design principles.**

I. Introduction

Large Language Model (LLM)–based Multi-Agent Systems (MAS) have become a foundational tool for complex problem solving in many areas such as software engineering [29], economic forecasting [6, 9], and scientific discovery [21, 23]. In these systems, performance depends not only on the base LLM, but critically on the workflow architecture: how agents are connected, how intermediate state is routed, and how role-specific prompts are structured [19]. Yet, designing multi-agent workflows remains largely guided by manual trial-and-error and informal "folk wisdom" (e.g., "more agents improve accuracy" or "multi-agent debate improves reasoning"). This raises the question: *Can workflow search traces be used to identify statistically reliable principles for multi-agent system design?*

Recent studies have taken initial steps toward quantifying MAS design choices, examining agent-count scaling [34], load balancing in compressor-predictor pipelines [14], and canonical topologies [19]. However, two bottlenecks limit existing paradigms: (1) *manual empirical studies* evaluate only a handful of fixed, hand-crafted topologies (typically 3–5 patterns), leaving composed workflows and sub-topological prompt implementations unexamined; and (2) *automated workflow optimizers* such as AFlow [33] and ADAS [15] dynamically search over code and prompts, but discard over 95% of evaluated architectures to return a single task-specific optimum W^*.

To bridge this gap, we introduce a two-phase empirical pipeline that transforms complete workflow-search trajectories into a controlled testbed for discovering design principles (Fig. 1). In the **Sampling Phase**, we repurpose the MCTS-based AFlow optimizer [33] to compose six workflow operators: Sequential, Star, SharedMemory, Debate, StarWithCoordinator, and DebateWithModerator. Rather than retaining only the final workflow, we collect all $N = 20$ evaluated workflows from each of $K = 5$ independent runs per condition. To limit confounding from external tools, specialized domain knowledge, and LLM-as-a-judge noise, we evaluate every workflow in a tool-free setting on 26 deterministically verified reasoning tasks from Enigmata [3]. This produces a corpus of 28,568 executable workflow–task–performance triplets.

In the **Analysis Phase**, we run an adapted AutoResearch loop [17] in a leakage-controlled sandbox with a strict search-level train/validation/test split. Across 300 trials, it mines features from orchestration code (`graph.py`) and role-prompt templates (`prompt.py`) under an L_0-regularized validation R^2 objective; we then test 70 candidate contrasts on held-out searches.

Our analysis identifies three effects that persist on held-out searches, spanning progressively finer levels of workflow specification: (1) *Entry-Point Topology:* workflows beginning with Star fan-out—in which parallel workers solve independently before a one-shot aggregator combines their outputs—perform worse than alternative entry structures (median $\Delta = -0.045$), corroborating Kim et al. [19]'s finding that uncoordinated agents can amplify errors; (2) *Communication State:* SharedMemory, in which workers sequentially update a persistent shared document [13], outperforms conversational message passing ($\Delta = +0.135$); and (3) *Prompt Structure:* more specific prompt templates and named placeholders that route context lift

[*]Equal contribution. XC: system implementation & operator set, large-scale experiments, data analysis, and empirical write-up; JC: project conception, theoretical framework & instruments, experimental design, and theoretical write-up.

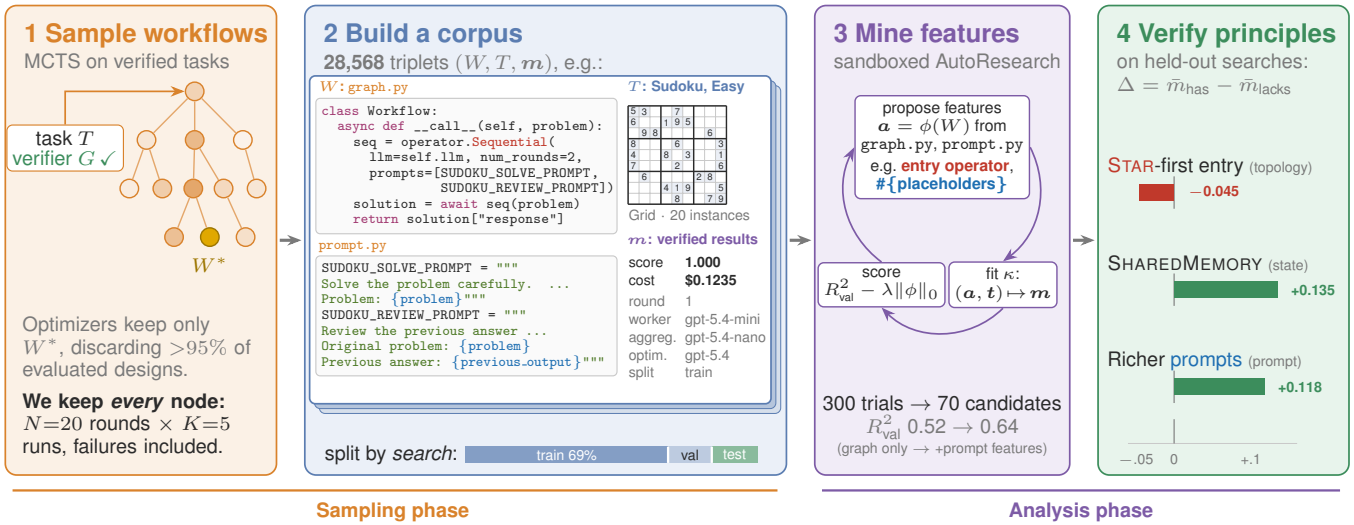


Fig. 1. **Overview of our two-phase pipeline. (1) Sample:** for each verified task bundle T, an AFlow MCTS search proposes and evaluates workflows; optimizers keep only the final W^*, whereas we keep *every* evaluated node (N=20 rounds × K=5 runs, failures included). **(2) Build a corpus:** each of the 28,568 entries is a triplet $(W, T, \boldsymbol{m})$, an executable workflow W (orchestration code `graph.py` plus its prompt templates `prompt.py`), a task bundle T, and verified metrics $\boldsymbol{m}$ (score, cost), split into train/val/test at the level of whole searches. **(3) Mine features:** a sandboxed AutoResearch loop proposes code and prompt features $\boldsymbol{a} = \phi(W)$ (e.g., the entry operator or the number of `{placeholders}`), fits a regressor κ from $(\boldsymbol{a}, \boldsymbol{t})$ to $\boldsymbol{m}$, and scores candidates by L_0-penalized validation R^2. **(4) Verify principles:** surviving features are tested on held-out searches as score contrasts Δ between workflows that have vs. lack the feature.

validation R^2 from 0.521 with graph features alone to 0.638, with held-out gains (Δ = +0.118) comparable to choosing the overall communication or orchestration topology.

Taken together, our work makes the follow three concrete contributions[1]:

1) **Full-Trace Design-Space Mining.** We introduce a pipeline that combines complete MCTS search trajectories with deterministic evaluation to mine code- and prompt-level design principles (§ III, § IV).
2) **Large-Scale Workflow Corpus.** We construct a corpus of 28,568 executable multi-agent workflows covering 26 verifiable reasoning tasks, with train/validation/test splits defined at the level of complete searches (§ V-A).
3) **Multi-Granularity Design Evidence.** We identify and validate held-out effects across three levels of workflow design: entry-point topology, communication state, and prompt structure (§ V-B).

II. Related Work

a) Multi-Agent System Design and Topologies: Designing effective multi-agent LLM workflows has largely relied on practitioner heuristics [5, 27]. While classical AI established blackboard architectures [13] where specialist modules read and write to a shared global state, recent LLM-MAS studies have begun quantifying specific design dimensions: Zhang et al. [34] and Qian et al. [24] studied agent-count and collaboration scaling; He et al. [14] analyzed load balancing in compressor-predictor pipelines; Chen et al. [4] examined diversity collapse; Willis et al. [28] studied large agent swarms; and Kim et al. [19] benchmarked five canonical MAS patterns. However, these works evaluate a small, fixed set of hand-crafted topologies and treat prompts as fixed constants. By contrast, we mine empirical regularities across tens of thousands of dynamically composed workflows, jointly analyzing macroscopic topologies and sub-topological prompt structure.

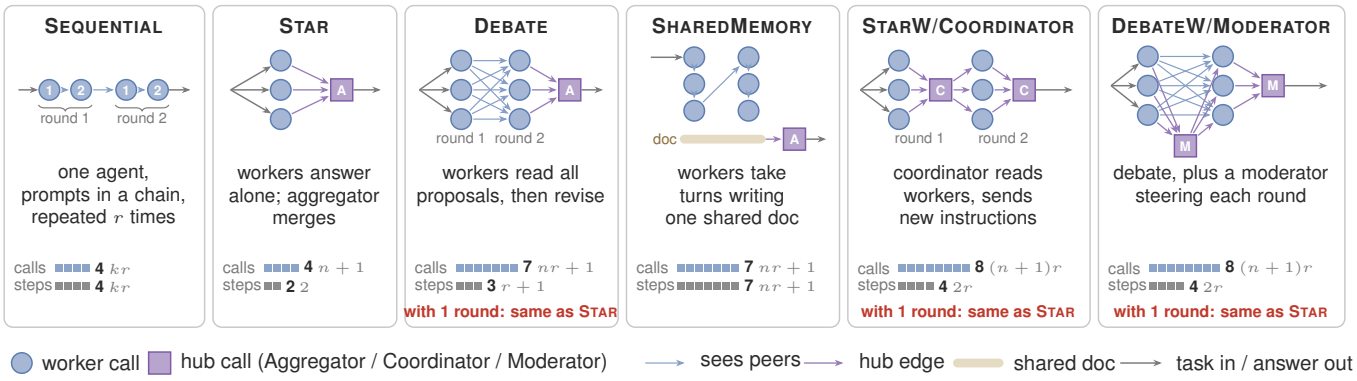
b) Automated Workflow and Prompt Search: Automated search methods have targeted prompt optimization [1, 12, 18], graph connections [31, 36], and agent code evolution [32]. Full-workflow optimizers, including MCTS-based AFlow [33], MASS [35], ADAS [15], and supernet routing [30], iteratively mutate workflow graphs and prompts to maximize accuracy. Whereas these methods discard intermediate candidates to output a single workflow W^*, we repurpose MCTS workflow search to harvest complete search trajectories for population-level analysis.

c) Controlled Reasoning Evaluation: Downstream agentic benchmarks in software engineering [16, 29], API/web interaction [10, 25], and scientific research [2] conflate multi-agent coordination with domain knowledge, external tool reliability, and LLM-as-a-judge variance [19]. To isolate reasoning and coordination, synthetic benchmarks use verifiable problems in constraint satisfaction [20], combinatorics [22], and SAT [26]. We evaluate workflows on Enigmata [3], leveraging its parametric generators and deterministic verifiers across difficulty tiers.

III. Formulating Design-Space Mining

Given a collection of executable multi-agent workflows, each evaluated on a concrete reasoning task T (e.g., a bundle of Sudoku instances checked by a verifier), we ask: *what structural*

[1]Our full code and dataset will be publicly released upon publication, along with all discovery artifacts.



Drawn for n=3 workers and r=2 rounds (SEQUENTIAL: k=2 prompts); a *round* is one full pass of the operator's communication pattern, repeated r times. **calls** = LLM calls (cost); **steps** = calls that must wait for an earlier one (latency if workers run in parallel). Every call also sees the original task. The optimizer chooses $n \in [1, 10]$, $r \in [1, 5]$, one prompt per worker and one hub prompt (`prompt.py`).

Fig. 2. **The six operator primitives O spanning our design space**, drawn for n=3 workers and r=2 rounds. Circles are worker calls and squares are hub calls (aggregator A, coordinator C, moderator M); blue edges show which earlier outputs a call reads, purple edges connect workers to a hub. SEQUENTIAL chains prompts through one agent; STAR fans out to isolated workers and aggregates once; DEBATE lets every worker read all peer proposals before revising; SHAREDMEMORY has workers take turns appending to one persistent document (blackboard); STARW/COORDINATOR and DEBATEW/MODERATOR add a hub call after every round. *calls* counts LLM calls (cost) and *steps* counts sequentially dependent calls (latency); with a single round the three multi-round operators reduce to STAR. The optimizer composes operators in `graph.py` and writes their prompts in `prompt.py` (Fig. 3).

properties of the workflow designs and prompts reliably predict higher task performance? In this section, we introduce our workflow representations and mining objective (§ III-A), as well as outline the three operational challenges motivating our pipeline in § III-B.

A. Workflow Representation and Mining Objective

Following Kim et al. [19] and Zhang et al. [33], we represent an evaluated agentic system as $S = (W, E)$, pairing an executable **workflow** $W \in \mathcal{W}$ with an **evaluation environment** $E = (T, G)$. We consider each part in turn.

a) Executable Workflows (W) and Environment (E): A workflow $W = (\mathcal{N}, C, \Omega)$ is operationalized as an executable Python program comprising two coupled source files: an orchestration graph (`graph.py`) and a prompt module (`prompt.py`). Here, $\mathcal{N} = \{v_1, v_2, \dots\}$ denotes the LLM-invoking agent calls, each parameterized by a backbone model M_i, a text prompt template P_i (with runtime placeholders such as `{problem}` or `{worker_outputs}`), and temperature τ_i. The communication topology C and message-routing policy Ω are expressed by composing multi-agent *operator primitives* from a vocabulary O (Fig. 2; parameters in Table VIII, § C-A). The environment $E = (T, G)$ pairs a *task bundle* T (i.e., *M problem instances from a given reasoning task and difficulty tier*) with a deterministic verifier G. Executing W on T yields a performance vector $\boldsymbol{m} = G(W, T) \in \mathcal{M}$, recording mean verification accuracy (score $\in [0, 1]$) and inference cost (USD).

b) Design-Space Feature Mining: In the *sampling phase*, we collect evaluated workflow triplets $\mathcal{D} = \{(W_i, T_i, \boldsymbol{m}_i)\}_{i=1}^{|\mathcal{D}|}$. Because each W_i is raw source code, in the *analysis phase* we search for a programmatic feature extractor $\phi : \mathcal{W} \to \mathbb{R}^d$ that parses `graph.py` and `prompt.py` into an interpretable feature vector $\boldsymbol{a} = \phi(W)$ (e.g., operator counts, entry-point topology, or prompt template length and placeholder count), alongside task descriptors $\boldsymbol{t} = \psi(T)$ and a regressor κ. Optimizing $(\phi, \psi, \kappa) \in \mathbf{H}$ on train/validation splits yields candidate features $\mathcal{H} \subset \mathbf{H}$. Each candidate is then verified on held-out test searches by splitting the workflows of every puzzle into two *arms*, as in an A/B test, a *high arm* in which the feature is present (or above the per-puzzle median) and a *low arm* in which it is absent (or at/below the median), and estimating the per-puzzle contrast $\Delta = \mathbb{E}[\text{score} \mid \text{high arm}] - \mathbb{E}[\text{score} \mid \text{low arm}]$ (§ V-B).

B. Methodological Challenges in Design-Space Mining

Extracting reliable empirical principles from observational workflow data requires overcoming three operational challenges (summarized in Table II and formalized in Appendix A):

a) Challenge 1: Design-Space Coverage vs. Implementation Competence: Mining principles requires a sample that both *covers* many distinct designs and consists of *competent* implementations. The naive route to coverage, generating workflow programs from scratch (e.g., prompting an LLM to write arbitrary orchestration code and prompts without constraints), fails the second requirement: most candidates break with syntax errors or degenerate prompts, burying topological effects under implementation noise. Conversely, keeping only an optimizer's final argmax workflow W^* discards the negative and intermediate examples needed to estimate contrasts. We therefore repurpose the MCTS-based AFlow optimizer (§ IV-A) over a structured 6-operator vocabulary O and retain *all* $N = 20$ intermediate workflows per search under a consistent LLM optimizer policy.

b) Challenge 2: Domain, Tool, and Grading Confounds: On downstream benchmarks such as software engineering or web navigation, accuracy is conflated with domain knowledge, external tool reliability, and LLM-as-a-judge grading noise [19]. To isolate multi-agent coordination and prompt structure, we evaluate workflows in a tool-free setting using Enigmata

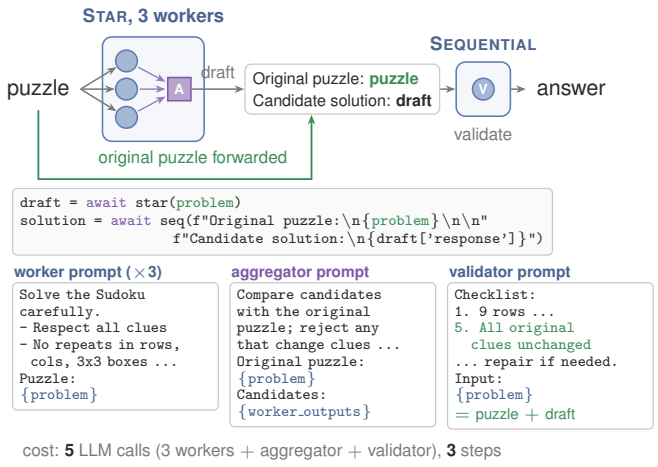


Fig. 3. **Composing operators in `graph.py`.** A STAR workflow (three workers, one aggregator) drafts a Sudoku solution; `graph.py` then forwards the *original* puzzle together with the draft into a one-step SEQUENTIAL validator. Every call's prompt template lives in `prompt.py`, with placeholders such as `{problem}` and `{worker_outputs}` filled at run time. Features such as the entry operator, chain length, and placeholder counts are parsed from exactly these two files.

(§ IV-B), where tasks are self-contained abstract reasoning puzzles scored by exact, deterministic rule-based verifiers on fixed instance bundles.

c) Challenge 3: High-Dimensional Feature Overfitting and Leakage: Because infinitely many features can be parsed from `graph.py` and `prompt.py`, manual feature selection is narrow, whereas unconstrained search can overfit or exploit dataset artifacts (such as MCTS round indices or within-search correlation). We address this via an L_0-regularized validation objective (§ IV-C) inside a strict sandbox, partitioning $\mathcal{D}$ into train, validation, and test splits at the *AFlow search level* and stripping all search/round identifiers from workflow files.

IV. METHODOLOGY: SAMPLING AND ANALYSIS PHASES

Our pipeline operates in two stages: a **sampling phase** that pairs full-trace AFlow workflow search (§ IV-A) with deterministic Enigmata evaluation (§ IV-B), followed by an **analysis phase** that mines workflow code and prompt features via a sandboxed AutoResearch loop (§ IV-C).

A. Sampling Phase: Full-Trace Workflow Search via AFlow

In AFlow [33], each workflow W is represented as an executable Python class in `graph.py` accompanied by role-specific hard text prompt templates in `prompt.py` (no soft prompts or gradient updates). Given a task bundle T, AFlow searches over workflows across $N = 20$ MCTS rounds: (i) *Selection:* A parent workflow is sampled from the top-k ($k = 4$) evaluated workflows using a mixed uniform/softmax distribution ($\lambda = 0.2$ uniform weight, temperature $\alpha = 0.4$; Eq. 4 in § C-B) to prevent premature collapse; (ii) *Expansion:* An LLM optimizer (fixed to `GPT-5.4` at temperature 0) inspects the parent's code and score history and proposes an edit that adds, removes, or rechains operators in `graph.py` and refines role prompts in `prompt.py` (§ D-D); (iii) *Evaluation:* The child workflow is executed on the $M = 20$ instances in T; and (iv) *Backpropagation:* Scores and costs are recorded in the search tree. Steps (i)–(iv) are the unmodified AFlow loop; we change three things (§ C-C). First, the operator registry is replaced by our six primitives (Fig. 2), so every proposed edit is a composition of known topologies. Second, evaluation targets fixed Enigmata bundles with deterministic verifiers (§ IV-B). Third, unlike point optimization that returns only W^*, we retain *all* $N = 20$ workflows per trace across $K = 5$ independent runs per condition, capturing both successful and failed designs below the best-so-far envelope (Fig. 4; single tree in Fig. 11, § H).

a) Composable Operator Primitives and Roles: To span established coordination patterns while permitting multi-stage compositions, we equip the search space with six operator primitives O (Fig. 2, Table VIII; § C-A), synthesized from recent MAS surveys [5, 19, 30, 35]: single-agent linear chaining (SEQUENTIAL), isolated parallel fan-out followed by one-shot aggregation (STAR), blackboard-style sequential writing to a persistent shared document (SHAREDMEMORY; 13), all-to-all peer proposal exchange (DEBATE), and hub-and-spoke or moderated multi-round variants (STARWITHCOORDINATOR, DEBATEWITHMODERATOR). An *operation* in `graph.py` is an asynchronous call to one operator primitive with optimizer-chosen parameters: $n \in [1, 10]$ workers, $r \in [1, 5]$ *rounds*, and custom prompts in `prompt.py`. A round is one complete pass of the operator's communication pattern, e.g., every SHAREDMEMORY worker reads the document once and appends to it, or every DEBATE worker reads its peers' proposals once and revises, and an operator with $r > 1$ repeats this pass r times, each pass conditioned on the previous one (Fig. 2 is drawn for r=2); these operator rounds are distinct from the N=20 MCTS *search* rounds above. Operations can be chained sequentially or conditionally, with the outputs of one operator (and the original problem) forwarded into the prompts of the next (e.g., STAR → SEQUENTIAL in Fig. 3; the most common chains are listed in Table XII). Each operator uses up to two LLM roles (a *worker* and an *aggregator/coordinator/moderator*) drawn from {`GPT-5.4-mini`, `GPT-5.4-nano`}, with at most 20 LLM calls per workflow execution (§ B).

B. Sampling Phase: Controlled Reasoning Evaluation via Enigmata

All agents operate in a *pure-reasoning, tool-free setting* (no code interpreters or search tools), so collaboration acts only through task decomposition, diverse candidate generation, peer verification, and shared state tracking. We select all 30 tasks in Enigmata [3] (*Crypto, Arithmetic, Logic, Grid, Graph, Search, and Sequential* categories) with programmatic generators and deterministic rule-based verifiers (exact match $\in \{0, 1\}$), yielding 87 (puzzle, difficulty) bundles across {Easy, Medium, Hard}. For each bundle T, we deterministically generate $M = 20$ instances and reuse this fixed set across all $K = 5$ searches and $N = 20$ rounds within that bundle so score differences reflect workflow structure rather than instance sampling noise (§ C-D). Filtering 1,031 syntax-invalid

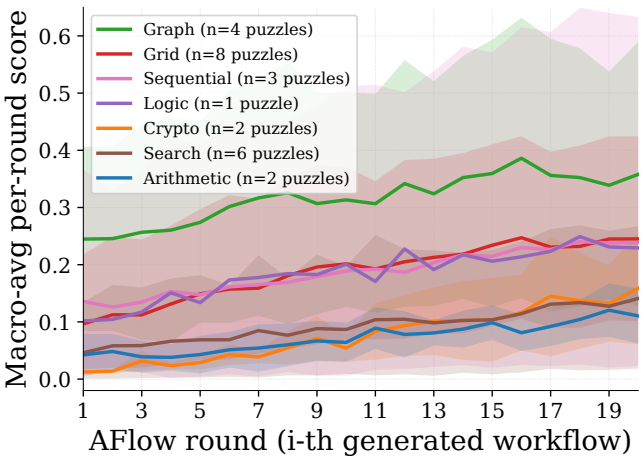


Fig. 4. **Full-Trace AFlow Search.** Macro-mean raw score per MCTS round by puzzle category (1,701 searches, 26 puzzles; 95% bootstrap CIs); retaining all 20 rounds captures a wide performance trajectory.

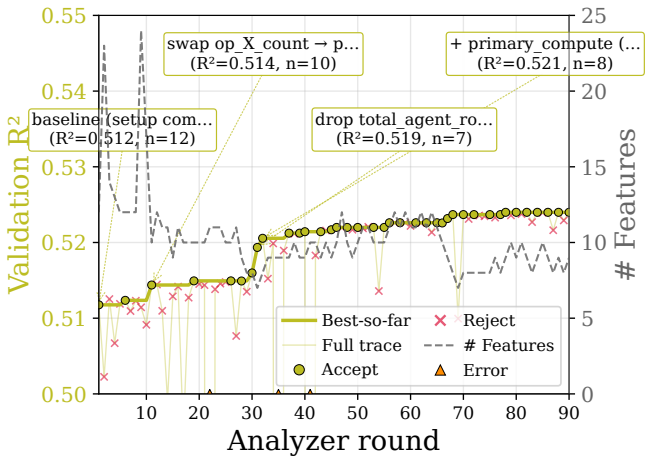


Fig. 5. **AutoResearch Feature-Mining Trace.** Validation R^2 (left axis) and active feature count $\|\phi\|_0$ (right axis) over iterations; the L_0-penalized objective improves fit while pruning redundant features.

workflows (3.0%) and excluding 4 sliding-tile puzzles where all single-agent baselines and workflows scored 0.00 (§ C-E) yields 28,568 executable workflows across 26 puzzles.

C. Analysis Phase: Sandboxed Feature Mining via AutoResearch

To extract design principles from `graph.py` and `prompt.py`, we adapt AutoResearch [17] as an automated feature-mining loop. At each iteration, an LLM agent edits a feature-extraction and regression script (`train.py`) that parses workflow source files into features $\phi(W) \in \mathbb{R}^d$, joins them with task descriptors $\psi(T)$, and fits per-puzzle regressors κ on D_{train}. To enforce parsimony and mitigate overfitting, each candidate is scored on the held-out validation split D_{val} via an L_0-regularized **validation objective** $\mathcal{F}(\phi, \psi, \kappa)$ defined as:

$$\begin{aligned} \mathcal{F}(\phi, \psi, \kappa) &= R^2_{D_{\text{val}}}(\boldsymbol{m}, \widehat{\boldsymbol{m}}) - \lambda \|\phi\|_0, \\ \widehat{\boldsymbol{m}} &= \kappa\big(\phi(W), \psi(T)\big). \end{aligned} \tag{1}$$

where $\|\phi\|_0 = d$ penalizes active feature count (Fig. 5; multi-run comparison in Fig. 7).

a) Search-Level Splitting and Sandbox Isolation: Because workflows within the same MCTS trace are evolutionary descendants, we partition the corpus into training (69.4%), validation (15.0%), and held-out test (15.6%) splits strictly at the *AFlow search level* (Table IX; § D-A0a). The feature-mining loop operates in a read-only sandbox that strips all search hashes, MCTS round indices, and file paths from workflow sources, forbids target-derived encodings, and physically isolates D_{test} for final verification (§ D-C).

V. Empirical Analysis

We first profile the corpus (coverage, sampling sufficiency, and gains over single-agent baselines; § V-A) and then present the design principles mined and verified at three levels of granularity (§ V-B).

A. Workflow Corpus and Coverage

Our corpus comprises 28,568 executable multi-agent workflows across 26 Enigmata puzzles (1,701 completed AFlow search traces out of 1,740 grid cells; Tables IX and XIII, § E), evaluated on 20-instance bundles at $0.0064 per puzzle instance (~ $4,454 total compute; Table X, § G). Workflows exercise all six operators individually and in chains: Sequential (47.2%), Star (44.5%), Debate (22.3%), SharedMemory (20.8%), DebateWithModerator (11.8%), and StarWithCoordinator (8.8%; Tables XI–XII).

a) Single-Agent Baselines, Coverage, and Sufficiency: Against one-shot evaluations of the single-agent backbones (`GPT-5.4`, `GPT-5.4-mini`, `GPT-5.4-nano`) on the identical puzzle bundles (Table XIV, § F), multi-agent workflows using only `mini`/`nano` workers substantially outperform even the stronger `GPT-5.4` baseline on tasks requiring iterative constraint propagation, lifting accuracy from 0.100 → 0.891 (mean) and 1.000 (max) on NaturalLanguageNavigation, 0.033 → 0.359 on MagicSquare, and 0.467 → 0.849 on StackPermutation. Full MCTS traces capture workflows spanning three orders of magnitude in cost ($0.01 to > $3.00 per bundle) and the full $[0, 1]$ accuracy range (Fig. 8), while the running cell mean converges within a 0.02 tolerance well before the full budget ($N_{\text{cfg}} = 59$ searches per puzzle; Fig. 9, Fig. 10).

B. Mining and Verifying Design Principles

a) Candidate Pool and Predictability Gating: We executed 10 main AutoResearch runs (`graph.py` only vs. `graph.py` + `prompt.py`; Fig. 7), totaling 300 automated feature-engineering trials (§ E-C). While `graph.py`-only feature sets plateaued at validation $R^2 = 0.521$ (8 features), jointly parsing `prompt.py` lifted validation R^2 to 0.638 (27 features), yielding 70 candidate binarized contrasts. Within each puzzle, workflows are split into high and low arms as defined in § III-A (≥ 20 workflows per arm) to estimate Δ with 95% cluster-robust Wald CIs clustered by AFlow search ID (§ E-B). To avoid diluting

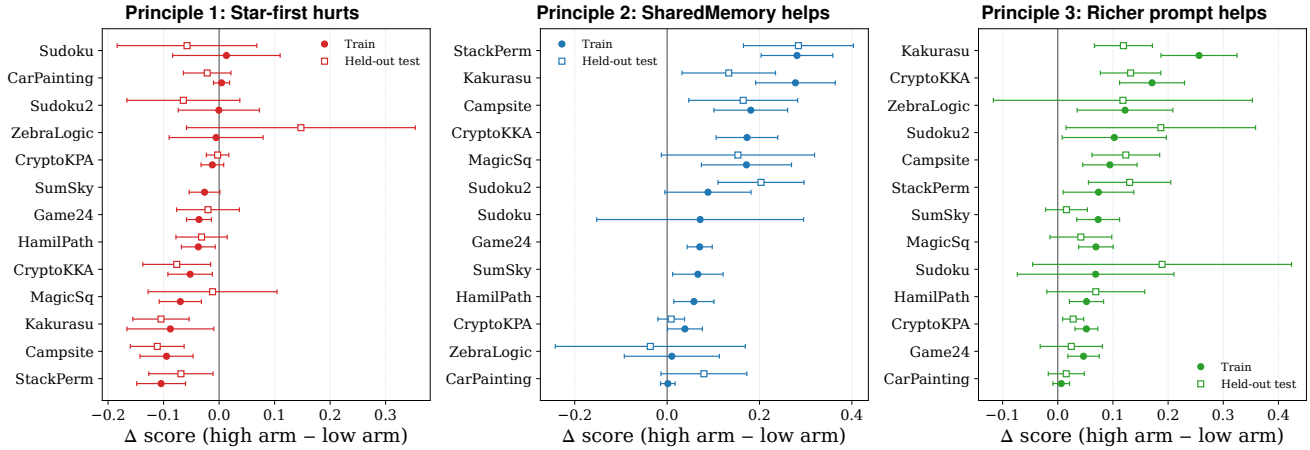


Fig. 6. **Per-Puzzle Effect Sizes of Mined Design Principles.** Per-puzzle score contrast Δ (high arm − low arm) for the three principles on the search-level train split (filled circles, n = 19,822) and held-out test split (open squares, n = 4,468), with 95% cluster-robust CIs; effect directions are consistent out-of-sample.

	Train					Held-out test					Sign agreement		Test-verified	
Variant	n	k_+	k_-	med Δ	p_W	n	k_+	k_-	med Δ	p_W	$k/n_\cap$	$p_B^\cap$	k/n_{tr}	p_B^{tr}
Principle 1 (RQ1, Entry-Point Topology): Star-first hurts performance *(claim direction −)*														
Star 1st	13	0	7	−0.036	.008	12	0	4	−0.045	.038	9/12	<.001	9/10	.011
Star×compute	13	0	7	−0.036	.008	12	0	3	−0.041	.038	9/12	<.001	9/10	.011
Principle 2 (RQ2, Communication State): SharedMemory boosts reasoning *(claim direction +)*														
SM present	13	10	0	+0.072	.002	10	6	0	+0.135	.011	9/10	<.001	9/10	.011
SM 1st	13	9	0	+0.072	.002	8	4	0	+0.143	.030	7/8	<.001	7/8	.035
Principle 3 (RQ3, Sub-Topological Prompts): Richer prompt structure helps *(claim direction +)*														
mean chars	13	11	0	+0.073	.002	13	6	0	+0.118	.002	13/13	<.001	13/13	<.001
max chars	13	10	0	+0.074	.002	13	7	0	+0.112	.002	13/13	<.001	13/13	<.001
#placeh.	13	9	0	+0.102	.002	13	7	0	+0.059	.003	11/13	<.001	11/13	.011

TABLE I

Statistical Summary of Mined Design Principles. Per variant: gate-admitted puzzles (n), cells with strictly positive (k_+) / negative (k_-) 95% cluster-Wald CIs, median effect size (Δ), and Wilcoxon p-values (p_W) on train and held-out test splits, with out-of-sample sign agreement ($p_B^\cap$) and conditional test verification (p_B^{tr}).

contrasts with floor/ceiling tasks, a pre-test *predictability gate* admits a puzzle iff a LightGBM surrogate trained on D_{train} achieves validation $R^2 > 0.05$ and Spearman $\rho > 0.30$ on D_{val} (§ I). Calibrated against a permutation null ($R^2_{p95} \le -0.06$, $\rho_{p95} \le 0.23$; Fig. 12), this gate admits 13 predictable puzzles prior to accessing D_{test} (Fig. 6, Table I; Table XV):

b) Principle 1 (RQ1, Entry-Point Topology): Star-first fan-out degrades reasoning: When a workflow begins with an isolated Star operator (parallel workers without lateral communication followed by one-shot aggregation), accuracy drops relative to alternative entry topologies on 7/13 train puzzles (median Δ = −0.036, p_W = .008), with 0/13 strictly positive cells (Fig. 6 left; Table V). This negative effect persists on the held-out test split (median Δ = −0.045, p_W = .038, 9/10 conditional sign replication), corroborating Kim et al. [19]: uncoordinated parallel fan-out amplifies reasoning errors that an aggregator must then reconcile without peer verification.

c) Principle 2 (RQ2, Communication State): SharedMemory outperforms conversational dialogue: Whereas LLM frameworks default to conversational message passing (Debate), workflows using blackboard-style SharedMemory [13], in which agents sequentially read and append to a persistent written document, outperform workflows without it on 10/13 train puzzles (median Δ = +0.072, p_W = .002) and 6/10 test puzzles (median Δ = +0.135, p_W = .011, 9/10 sign agreement; Fig. 6 middle, Table VI), achieving the highest corpus-wide mean accuracy of all six operators (0.23 vs. 0.18 for Debate and 0.16 for Star; Table XI), showing that persistent written state mitigates "lossy communication" in multi-round dialogue [19].

d) Principle 3 (RQ3, Sub-Topological Prompt Structure): prompt specificity and placeholder routing predict accuracy: Below the categorical topology grain, workflows with above-median prompt template character length (`mean chars` in `prompt.py`) achieve strictly positive accuracy gains on 11/13 train (median Δ = +0.073, p_W = .002) and 6/13 test puzzles (median Δ = +0.118, p_W = .002, 13/13 sign agreement; Fig. 6 right, Table VII). Similarly, the count of `{name}` placeholders (`#placeh.`) replicates this positive effect on 9/13 train (median Δ = +0.102) and 7/13 test puzzles (median Δ = +0.059, 11/13 sign agreement). We do not claim character length is causal in isolation; rather, sub-topological prompt specification and context routing carry effect sizes (Δ ≈ +0.06 to + 0.12) on par with macro-topology selection,

which is why `prompt.py` features lift validation R^2 from 0.521 to 0.638 (Fig. 7).

e) Out-of-Sample Verification and Test Precision: Across all seven variants in Table I, two-sided Wilcoxon tests ($p_W < 0.05$), binomial sign-agreement tests ($p_B^{\cap} \leq .001$), and conditional test-verification tests ($p_B^{tr} \leq .035$) confirm directional reproducibility on D_{test}. Median effect sizes hold on test, but fewer puzzles clear strict 95% cluster-Wald CIs (k_+/k_-) than on train (e.g., 4/12 vs. 7/13 for Star-first; 6/10 vs. 10/13 for SharedMemory) because the 15.6% test split has fewer search clusters ($G \in [4, 14]$ vs. $[19, 47]$; § E-B): the findings are directionally robust, with finite-sample uncertainty on task-specific magnitudes.

VI. Discussion and Future Directions

a) Full-Trace Mining vs. Point Optimization: Automated workflow search and empirical design-space science are complementary: the non-argmax rounds that search discards provide exactly the negative and intermediate comparisons needed to explain why designs fail and which edits help. Full traces also expose accuracy-cost trade-offs: SharedMemory costs more than Debate or Star ($0.25 vs. $0.13; Table XI) as the shared document grows, yet its persistent written state avoids the information loss of multi-round dialogue and yields the highest mean accuracy in the corpus.

b) Beyond Topology-Only Benchmarks: Macro-topology alone (`graph.py`) explains only part of workflow variance ($R^2 = 0.521$); jointly mining sub-topological prompt structure (`prompt.py`) lifts out-of-sample predictive power to $R^2 = 0.638$ (§ V-B). Topology benchmarks should therefore report and control for prompt template richness and placeholder routing rather than treating prompts as fixed constants: an ostensibly superior communication graph can be bottlenecked by underspecified role instructions or missing context placeholders.

c) Operator Evolution and Tool-Rich Domains: Feeding mined regularities back into the operator registry could guide the synthesis of new operator primitives [8, 23] or prune suboptimal entry-point patterns (such as uncoordinated Star-first fan-out) from the optimizer's search prior, and extending the pipeline to tool-rich domains such as coding [16], games [11], and social simulations [7] will test how topological and prompt principles interact with external tool use and environment feedback.

VII. Conclusion

We presented an end-to-end pipeline that moves multi-agent LLM workflow design from manual "folk wisdom" toward systematic statistical mining of design principles. Full-trace AFlow search over 26 deterministically verified Enigmata tasks yielded 28,568 executable workflows, and a sandboxed, search-level-split AutoResearch loop over 300 trials surfaced out-of-sample verified principles at three granularities: Star-first fan-out degrades accuracy, blackboard-style SharedMemory outperforms conversational debate, and prompt template specificity and placeholder routing matter as much as macroscopic topology. We release the workflow corpus, evaluation harness, and sandboxed analysis pipeline for reproducible empirical research on multi-agent systems.

Limitations

To isolate multi-agent coordination and prompt structure from external confounds, our experiments focus on tool-free, deterministically verified reasoning tasks from Enigmata and evaluate workflows within a single model family (`GPT-5.4` as the MCTS optimizer and `{GPT-5.4-mini, GPT-5.4-nano}` as worker and aggregator backbones; § B). While this controlled setup eliminates grading noise and API variance (Appendix A), extending our pipeline to open-weight model families, alternative workflow search algorithms, and tool-interactive domains (such as repository-level coding or web navigation) is a natural direction for future work. Finally, because our corpus harvests trajectories generated along MCTS optimization runs, the mined principles reflect structural regularities within the space of executable workflows explored by the optimizer rather than unconstrained random code generation.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=RQm2KQTM5r.

[2] Jonathan Bragg, Mike D'Arcy, Nishant Balepur, Dan Bareket, Bhavana Dalvi Mishra, Sergey Feldman, Dany Haddad, Jena D. Hwang, Peter Jansen, Varsha Kishore, Bodhisattwa Prasad Majumder, Aakanksha Naik, Sigal Rahamimov, Kyle Richardson, Amanpreet Singh, Harshit Surana, Aryeh Tiktinsky, Rosni Vasu, Guy Wiener, Chloe Anastasiades, Stefanus Candra, Jason Dunkelberger, Daniel Emery, Rob Evans, Malachi Hamada, Regan Huff, Rodney Kinney, Matt Latzke, Jaron Lochner, Ruben Lozano-Aguilera, Ngoc-Uyen Nguyen, Smita Rao, Amber Tanaka, Brooke Vlahos, Peter Clark, Doug Downey, Yoav Goldberg, Ashish Sabharwal, and Daniel S Weld. Astabench: Rigorous benchmarking of AI agents with a scientific research suite. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=M7TNf5J26u.

[3] Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Jiaze Chen, Xuefeng Li, Qiying Yu, Hao Zhou, and Mingxuan Wang. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=fmnxunacr4.

[4] Nuo Chen, Yicheng Tong, Yuzhe Yang, Yufei He, Xueyi Zhang, Zou Qingyun, Qian Wang, and Bingsheng He. Diversity collapse in multi-agent llm systems: Structural coupling and collective failure in open-ended idea generation. In *Findings of The 64th Annual Meeting of the Association for Computational Linguistics*, 2026.

[5] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=EHg5GDnyq1.

[6] Junyan Cheng and Peter Chin. Empirical asset pricing with large language model agents. *arXiv preprint arXiv:2409.17266*, 2024.

[7] Junyan Cheng and Peter Chin. Sociodojo: Building lifelong analytical agents with real-world text and time series. In *The twelfth international conference on learning representations*, 2024.

[8] Junyan Cheng, Peter Clark, and Kyle Richardson. Language modeling by language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[9] Junyan Cheng, Kyle Richardson, and Peter Chin. Analytica: Soft propositional reasoning for robust and scalable llm-driven analysis. In *The Fourteenth International Conference on Learning Representations*, 2026.

[10] Junyan Cheng, Ankit Srivastava, Jessie Zeng, Milenko Drinic, and Jack Stokes. Apeiron: A scalable LLM-agentic framework for autonomous full-lifecycle demand-optimized application synthesis. In *Findings of The 64th Annual Meeting of the Association for Computational Linguistics*, 2026.

[11] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35:18343–18362, 2022.

[12] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ZG3RaNIsO8.

[13] Barbara Hayes-Roth. A blackboard architecture for control. *Artificial Intelligence*, 26(3):251–321, 1985.

[14] Shizhe He, Avanika Narayan, Ishan S Khare, Scott W Linderman, Christopher Ré, and Dan Biderman. An information theoretic perspective on agentic system design. *arXiv preprint arXiv:2512.21720*, 2025.

[15] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=t9U3LW7JVX.

[16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[17] Andrej Karpathy. AutoResearch, 2026. URL https://github.com/karpathy/autoresearch.

[18] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[19] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Mark Malhotra, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025.

[20] Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. Zebralogic: On the scaling limits of llms for logical reasoning. In *International Conference on Machine Learning*, pages 37889–37905. PMLR, 2025.

[21] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of ai research. *Nature*, 651(8107):914–919, 2026.

[22] Chinmay Mittal, Krishna Kartik, Parag Singla, et al. Fcorebench: Can large language models solve challenging first-order combinatorial reasoning problems? *arXiv preprint arXiv:2402.02611*, 2024.

[23] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[24] Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large language model-based multi-agent collaboration. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=K3n5jPkrU6.

[25] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

[26] Anjiang Wei, Yuheng Wu, Yingjia Wan, Tarun Suresh, Huanmi Tan, Zhanke Zhou, Sanmi Koyejo, Ke Wang, and Alex Aiken. Satbench: Benchmarking llms' logical reasoning via automated puzzle generation from sat formulas. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33820–33837, 2025.

[27] Julia Wiesinger, Patrick Marlow, and Vladimir Vuskovic. Agents. White paper, Google, September 2024.

[28] Richard Willis, Jianing Zhao, Yali Du, and Joel Z Leibo. Evaluating collective behaviour of hundreds of llm agents. *arXiv preprint arXiv:2602.16662*, 2026.

[29] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[30] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, LEI BAI, and Xiang Wang. Multi-agent architecture search via agentic supernet. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=imcyVlzpXh.

[31] Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=LpE54NUnmO.

[32] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=pUpzQZTvGY.

[33] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=z5uVAKwmjf.

[34] Qin Zhang, Yangbin Yu, Qiang Fu, Deheng Ye, et al. More agents is all you need. *Transactions on Machine Learning Research*, 2024.

[35] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö Arık. Multi-agent design: Optimizing agents with better prompts and topologies. *arXiv preprint arXiv:2502.02533*, 2025.

[36] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=uTC9AFXIhg.

Appendix A
Methodological Formalization and Statistical Estimands

As introduced in § III, our pipeline mines empirical design principles from observational workflow trajectories $\mathcal{D} = \{(W_i, T_i, \boldsymbol{m}_i)\}_{i=1}^{|\mathcal{D}|}$. Here we formalize the statistical estimands recovered by our approach and clarify how each stage (AFlow traversal, Enigmata verification, and search-level sandboxed AutoResearch) addresses key threats to validity when learning from non-randomized workflow corpora; Table II maps each challenge to the pipeline component that addresses it.

A. Implementation Variance and Policy-Conditioned Sampling

When mapping a raw workflow W (consisting of `graph.py` and `prompt.py`) into an interpretable feature vector $\boldsymbol{a} = \phi(W)$, microscopic implementation details $\theta_s \in \Theta_s$ (such as exact phrasing choices or variable names not captured by ϕ) remain unmodeled. Marginalizing over the distribution of implementation details under a sampling policy $\mathcal{L}$, the expected performance of workflows with feature profile $\boldsymbol{a}$ on task $\boldsymbol{t}$ is:

$$\mu_{\mathcal{L}}(\boldsymbol{a}, \boldsymbol{t}) = \int_{\Theta_s} \mathbb{E}[\boldsymbol{m} \mid \boldsymbol{a}, \boldsymbol{t}, \theta_s]\, P_{\mathcal{L}}(\theta_s \mid \boldsymbol{a}, \boldsymbol{t})\, d\theta_s \quad (2)$$

a) Why MCTS Trajectory Harvesting Beats Naive Random Code Sampling: If $\mathcal{L}$ were a uniform random code-and-string generator over Θ_s, the vast majority of sampled workflows would contain syntax errors or incoherent prompts, causing extreme variance in θ_s to swamp structural differences in $\boldsymbol{a}$. Conversely, retaining only the final argmax W^* from an optimizer collapses support onto a single point per search. By harvesting all $N = 20$ MCTS rounds generated under a fixed LLM optimizer (`GPT-5.4` at temperature 0) with a mixed uniform/softmax parent-selection policy ($\lambda = 0.2$; Eq. 4), AFlow restricts Θ_s to syntactically valid, competently prompted implementations while preserving broad variation across operators and prompt templates. Importantly, as noted in our Limitations, $\mu_{\mathcal{L}}(\boldsymbol{a}, \boldsymbol{t})$ represents a *policy-conditioned observational estimand* relative to the foundation model's capability frontier rather than an unconditioned interventional effect.

B. Evaluation Variance and Fixed-Bundle Verification

Let $G(W, T)$ denote the observed evaluation score of workflow W on task bundle $T = \{x_1, \ldots, x_M\}$. In open-ended benchmarks that rely on external tool APIs, web state, or LLM-as-a-judge grading, the observed score decomposes as:

$$\widehat{\boldsymbol{m}}(W, T) = \mu_{\text{reasoning}}(W, T) + \xi_{\text{domain/tool}} + \epsilon_{\text{judge}} \quad (3)$$

where $\xi_{\text{domain/tool}}$ reflects external API or domain-knowledge bottlenecks and ϵ_{judge} represents grading noise.

a) Deterministic Verification on Reused Bundles (Enigmata): Evaluating tool-free workflows on Enigmata [3] eliminates subjective judge variance ($\epsilon_{\text{judge}} = 0$) because every puzzle instance is scored by an exact, deterministic rule-based verifier. Furthermore, because the same fixed bundle of $M = 20$ deterministically generated instances is reused across all $K = 5$ searches and $N = 20$ MCTS rounds within each (puzzle, difficulty) cell (§ C-D), instance-sampling variance between workflows compared within the same cell cancels out in paired per-puzzle contrasts $\Delta = \mathbb{E}[\text{score} \mid \text{high arm}] - \mathbb{E}[\text{score} \mid \text{low arm}]$.

C. Overfitting, Trajectory Leakage, and Out-of-Sample Verification

Let $S(\phi) \in \{0, 1\}$ denote the indicator that a candidate feature extractor $\phi \in \mathbf{H}$ is selected during feature mining. Two forms of overfitting threaten automated feature discovery over workflow code: (1) *within-search trajectory leakage*, where correlated MCTS rounds from the same search straddle training and evaluation splits; and (2) *test-set selection leakage*, where feature selection $S(\phi)$ conditions on held-out test outcomes $\boldsymbol{m}_{\text{test}}$.

a) Search-Level Splitting and Pre-Specified Test Verification: Our analysis phase prevents both failure modes structurally. First, partitioning $\mathcal{D}$ into D_{train}, D_{val}, and D_{test} at the *AFlow search level* guarantees that all $N = 20$ rounds of any search trajectory reside in a single split. Second, the AutoResearch loop evaluates the L_0-penalized objective $\mathcal{F}(\phi, \psi, \kappa)$ (Eq. 1) exclusively on D_{val} inside an isolated sandbox with no access to D_{test} or row/round identifiers. Because both the candidate features ϕ and the hypothesized contrast directions are fixed prior to accessing D_{test}, the selection mechanism satisfies conditional independence $S(\phi) \perp\!\!\!\perp \boldsymbol{m}_{\text{test}} \mid \boldsymbol{a}$, ensuring that test-split contrasts provide a valid, uncontaminated out-of-sample replication check.

Appendix B
Experiment Details and Hyperparameters

This section consolidates the configuration choices that govern the sampling and analysis phases. Hyperparameters are grouped by component, model selection, workflow constraints, the AFlow search, and the predictability gate (Table III). Table IV pins the OpenAI model snapshots used throughout.

a) Model selection: The Optimizer role, which proposes structural workflow modifications during AFlow search, is fixed to GPT-5.4 in order to maintain a single, stationary generative policy across the corpus (cf. § A). The Worker and Aggregator roles are drawn from {GPT-5.4-mini, GPT-5.4-nano}, providing a controlled 2×2 contrast between worker and aggregator capability while keeping the per-search call budget tractable.

b) Workflow constraints: Each generated workflow is capped at 20 atomic LLM calls, which bounds the per-evaluation cost while still admitting compositions of multiple operators (§C-A). Because each workflow is evaluated on the full 20-instance bundle, this caps the per-search call budget at $20 \times 20 = 400$ atomic calls per round and at most 8,000 calls for an entire AFlow search trace. The operator set comprises the six primitives listed in Table VIII.

c) AFlow search: Each (puzzle, bundle, model) cell is searched $K = 5$ times for $N = 20$ MCTS rounds. The selection mixture (Eq. 4) uses $\lambda = 0.2$ to balance uniform exploration against score-weighted exploitation, with score temperature $\alpha = 0.4$ and a top-k pool of size $k = 4$.

Challenge	Manifestation in MAS Evaluation	Pipeline Stage	Methodological Design
Design-space coverage & implementation variance	Manual studies test only a few fixed topologies; argmax search discards intermediate designs; naive random code generation produces broken prompts or syntax errors.	AFlow Search (*Sampling*; § IV-A)	Retains *all* $N = 20$ MCTS rounds across $K = 5$ runs over 6 composable operators, sampling both failed and high-performing workflows under a consistent LLM optimizer policy (§ A).
Domain & evaluation confounds	Downstream benchmarks conflate coordination topology with external tool APIs, domain knowledge, or noisy LLM-as-a-judge grading.	Enigmata Verification (*Sampling*; § IV-B)	Tool-free evaluation on 26 parametric abstract-reasoning tasks with exact, deterministic rule-based verifiers and fixed 20-instance bundles per difficulty tier (§ C-D).
High-dimensional feature overfitting & leakage	Workflow code (`graph.py`) and prompts (`prompt.py`) admit infinitely many features; manual selection is narrow, while unconstrained search overfits or leaks test data.	AutoResearch Mining (*Analysis*; § IV-C)	Sandboxed AST/prompt feature mining with an L_0-penalized validation R^2 objective on strict search-level splits; pre-specified directional verification on held-out test searches (§ D).

TABLE II

Core methodological challenges in empirical MAS design-space mining and corresponding pipeline components. Each stage of the pipeline addresses a specific operational threat to extracting reproducible design principles.

Hyperparameters	Values
Models	
Optimizer	GPT-5.4
Opt. Temperature (t)	0
Worker	[GPT-5.4-mini, GPT-5.4-nano]
Worker temperature (t)	0
Aggregator	[GPT-5.4-mini, GPT-5.4-nano]
Aggregator temperature (t)	0
Workflow Constraints	
Maximal number of calls per workflows	20
# Operators	6
AFlow	
Number of evaluation step per round	1
Number of search steps	20
Number of searches per experimental condition	5
Number of difficulty bundles	2,3
Number of puzzles per bundle	20
Exploration/Exploitation weight (λ)	0.2
Top-k score-based selection	4
Score influence factor λ	0.4
Predictability gating	
R^2 threshold	0.05
ρ threshold	0.30
Min entry per cell	20
Model	LightGBM
Learning rate	0.05
Number of leaves	31
Min child samples	10

TABLE III
Hyper-parameters

Model name	Snapshot
GPT-5.4	gpt-5.4-2026-03-05
GPT-5.4-mini	gpt-5.4-mini-2026-03-17
GPT-5.4-nano	gpt-5.4-nano-2026-03-17

TABLE IV
Model snapshots for the OpenAI models.

d) Predictability gate: The LightGBM surrogate (200 trees, learning rate 0.05, 31 leaves, `min_child_samples` = 10, fixed seed) used for the predictability gate admits a puzzle if $R^2 > 0.05$ *and* Spearman $\rho > 0.30$ on its validation rows; the calibration of these thresholds against a permutation null is detailed in §I. The minimum cell occupancy of 20 workflows in either arm is enforced for every per-puzzle binarized contrast.

A. Enigmata puzzles

We selected a subset of the Enigmata puzzles, namely we focused on 30 puzzles for which the code-base provided both generator and verifiers, these puzzle collection covers 2 Arithmetic, 2 Crypto, 4 Graph, 8 Grid, 1 Logic, 6 Search and 7 Sequential puzzles (Table XIII).

a) Puzzle Search space: Our sampling grid varies seven factors: (1) puzzle type, (2) difficulty bundle, (3) optimizer model, (4) worker model, (5) aggregator model, (6) AFlow search run, and (7) MCTS iteration. Detailed information about these factors are available in Table III.

1) **Puzzle type:** $p = 30$ Enigmata puzzles, e.g., Sudoku, TicTacToe, ZebraLogic.
2) **Difficulty bundles:** *easy*/*medium*/*hard*, where 27 puzzles expose all three ($b = 3$) and 3 puzzles, Hitori, LightUp, TicTacToe, only *medium*/*hard* ($b = 2$). A bundle is a collection of puzzle instance associated with these difficulties.
3) **Optimizer model:** $o = 1$ the LLM backbone that proposes workflow updates in AFlow, e.g., GPT-5.4.
4) **Worker model:** $w = 2$, LLM backbones running individual task work, the Star-operator leaves.
5) **Aggregator model:** $g = 2$, LLM backbones handling operator-level synthesis.
6) **AFlow search run:** $K = 5$ AFlow searches per (puzzle, bundle, model) cell.
7) **AFlow search execution:** $N = 20$ MCTS rounds per AFlow search.

For each (puzzle, bundle) pair we generate 20 puzzle instances deterministically from their associated Enigmata difficulty level.

A naive grid with uniform $b = 3$ would yield $30 \cdot 3 \cdot 1 \cdot 2 \cdot 2 \cdot 5 \cdot 20 = 36{,}000$ workflows from 1,800 searches. Nonetheless, given the three puzzles missing *easy* there are $27 \cdot 3 + 3 \cdot 2 = 87$ (puzzle, bundle) cells and $87 \cdot 1 \cdot 2 \cdot 2 \cdot 5 = 1{,}740$ searches, leading to at most 34,800 workflows. Out of the possible 1,740 searches, 1,701 completed successfully (37 mid-search aborts and 2 dispatcher misses, uniformly spread across the grid), producing $1{,}701 \cdot 20 = 34{,}020$ (97.7%) candidate workflows.

b) Executable workflow filtering: From these 34,020 workflows we further filter 1,031 (3%) for which the graphs are either missing or could not be evaluated, for instance because a custom prompt generated by the AFlow Optimizer model was undefined or mis-referenced.

c) Call-budget constraint: Each workflow is authorized a maximal number of atomic LLM calls of 20, given the type of puzzles, this still allows workflows that contain multiple operators with few calls or operators that leverage single operators with many calls. Given that each workflow is evaluated on all the puzzles of a bundle, this leads to 20×20 calls per AFlow search round, and 8,000 calls at most for a single AFlow search.

d) Consistent Sequential Puzzle failures: After further analyses, we removed 4 of the Sequential puzzles: EightPuzzle, NinePuzzle, FifteenPuzzle, SixteenPuzzle. Their minimal and maximal scores remained at 0 across all searches, from further inspection the LLM answers were either wrong attempts or simply stated that there was no solution to the puzzles. Following this removal step we obtain a reduced set of 28,568 workflows.

1) Feature-Mining Search Evolution: We conduct multiple AutoResearch runs with varying prompts, aiming at optimizing the validation set R^2 while keeping the number of features small. We considered both validation R^2 over the entire population, and average R^2 reported across per-puzzle split. These searches keep track of the evolution of the validation R^2, features, and size of the feature set, allowing us to restore a search step and leverage discovered feature and combination of features, Fig. 7 shows an example of such traces.

B. Analysis Phase

Each candidate design principle is evaluated per puzzle. For a given feature, workflows are partitioned within each puzzle into a *high arm* and a *low arm*: for binary features (e.g., presence of a SharedMemory operator) the high arm consists of workflows where the feature is present and the low arm those where it is absent; for count or continuous features (e.g., mean prompt-template character length) the partition is the per-puzzle median split, with the high arm strictly above the median and the low arm at or below. The per-puzzle effect size is the difference in mean score between the two arms, $\Delta = \mathbb{E}[\text{score} \mid \text{high arm}] - \mathbb{E}[\text{score} \mid \text{low arm}]$, and per-puzzle verdicts are aggregated to a principle-level verdict.

a) Predictability gate: For each puzzle, we fit a LightGBM regressor on the training-split workflows (200 trees, learning rate 0.05, 31 leaves, `min_child_samples` = 10, fixed seed) predicting score from design features, and evaluate it on the puzzle's held-out validation workflows. A puzzle is admitted iff validation $R^2 > 0.05$ *and* Spearman $\rho > 0.30$. Both are required: a constant predictor attains $R^2 = 0$ with undefined ρ, while a rank-preserving but magnitude-compressing predictor attains $\rho > 0$ with $R^2 \leq 0$, so the conjunction enforces both calibration and ranking (see § I).

b) Feature-set for the predictability gate: The predictability gate is parameterized by the feature set supplied to the regression surrogate, since the admission of a puzzle is meaningful only if the surrogate has access to features of the type the candidate principle concerns. A puzzle whose validation R^2 exceeds the gate threshold under a feature set that includes prompt-length statistics provides no evidence that its score variation is explained by structural design choices; conversely, restricting the surrogate to structural features alone risks excluding puzzles whose design signal is real but weaker than a prompt signal.

We build the feature set using both handcrafted features that leverage the actual execution code of the workflows, associated external prompt files, and features that were extracted during AutoResearch explorations.

1) Per-Puzzle Breakdowns of Mined Principles: Tables V–VII break down the headline results of Table I by puzzle, one table per principle. Each row gives the contrast Δ of a gate-admitted puzzle on the train (tr) and held-out test (te) splits; a dash marks a cell that is not testable on that split (fewer than 20 workflows in an arm). The bottom rows report how many testable cells have a 95% cluster-Wald CI strictly above (k_+/n) or below (k_-/n) zero, and the median Δ.

Appendix C
Design Space Specifications

We designed the operator set based on an extensive literature review covering contemporary research papers [5, 15, 24, 30, 35], agentic system design whitepapers, and guidelines from popular GitHub repositories. We cover most of the designs from [19], with the exception of the Hybrid. Our set of operators covers commonly used patterns; our aim was to provide the general topologies, while leveraging AFlow ability to combine workflows, so that the core topological components could be assembled into complex patterns while making it possible to track operator usage.

A. Operator Set

The operator set is the unit of architectural variation in our search space. Each operator is a multi-agent *topology primitive* whose call signature exposes a small set of structural knobs (number of agents, number of rounds, prompts) and consumes one or two of the three LLM roles: *worker*, *aggregator* (or its role-equivalents *coordinator*, *moderator*), and *optimizer*. The six operators currently registered are listed below; Fig. 2 draws each of them for $n = 3$ workers and $r = 2$ rounds

TABLE V

PRINCIPLE 1 (RQ1): STAR AS THE FIRST OPERATOR LOWERS SCORE. PER-PUZZLE EFFECT SIZE Δ SCORE (HIGH ARM − LOW ARM) ON THE TRAIN AND HELD-OUT TEST SPLITS. FEATURE DEFINITIONS: **STAR 1ST** = BINARY INDICATOR: 1 IFF THE FIRST OPERATOR INSTANTIATED IN `graph.py` (PARSED IN SOURCE ORDER) IS `Star`; HIGH ARM = 1, LOW ARM = 0. k_+/n AND k_-/n COUNT CELLS WHOSE 95% CLUSTER-WALD CI IS STRICTLY ABOVE / BELOW ZERO (OUT OF TESTABLE CELLS). HEADLINE: STAR 1ST $-7/13$ ON TRAIN (MEDIAN $\Delta = -0.036$); STAR 1ST $-4/12$ ON TEST (MEDIAN $\Delta = -0.045$).

	Star 1st	
Puzzle	tr	te
Campsite	−0.095	−0.111
CarPainting	+0.004	−0.021
CryptoKKA	−0.052	−0.076
CryptoKPA	−0.012	−0.003
Game24	−0.036	−0.020
HamiltonianPath	−0.037	−0.032
Kakurasu	−0.088	−0.105
MagicSquare	−0.070	−0.012
StackPermutation	−0.104	−0.069
Sudoku	+0.013	−0.058
Sudoku2	−0.000	−0.064
SumSkyscraper	−0.026	–
ZebraLogic	−0.005	+0.147
k_+/n	0/13	0/12
k_-/n	7/13	4/12
median Δ	−0.036	−0.045

TABLE VI

PRINCIPLE 2 (RQ2): SHAREDMEMORY OPERATORS RAISE SCORE. PER-PUZZLE EFFECT SIZE Δ SCORE (HIGH ARM − LOW ARM) ON THE TRAIN AND HELD-OUT TEST SPLITS. FEATURE DEFINITIONS: **SM PRESENT** = COUNT OF SHAREDMEMORY OPERATOR INSTANTIATIONS IN `graph.py`, MEDIAN-SPLIT WITHIN EACH PUZZLE; THE PER-PUZZLE MEDIAN IS 0 FOR EVERY PUZZLE SO THE HIGH ARM = WORKFLOWS WITH $\geq$1 SHAREDMEMORY OPERATOR AND THE LOW ARM = WORKFLOWS WITH NONE. k_+/n AND k_-/n COUNT CELLS WHOSE 95% CLUSTER-WALD CI IS STRICTLY ABOVE / BELOW ZERO (OUT OF TESTABLE CELLS). HEADLINE: SM PRESENT +10/13 ON TRAIN (MEDIAN Δ = +0.072); SM PRESENT +6/10 ON TEST (MEDIAN Δ = +0.135).

	SM present	
Puzzle	tr	te
Campsite	+0.168	+0.178
CarPainting	+0.003	+0.076
CryptoKKA	+0.178	+0.139
CryptoKPA	+0.036	+0.009
Game24	+0.072	–
HamiltonianPath	+0.055	+0.055
Kakurasu	+0.282	+0.131
MagicSquare	+0.172	+0.161
StackPermutation	+0.273	+0.294
Sudoku	+0.010	–
Sudoku2	+0.107	+0.161
SumSkyscraper	+0.068	–
ZebraLogic	+0.012	−0.051
k_+/n	10/13	6/10
k_-/n	0/13	0/10
median Δ	+0.072	+0.135

together with its call and step counts, and Fig. 3 shows how two operators are chained inside `graph.py`. All extend a common base class that clamps the multi-agent knobs to safe ranges: $1 \leq$ `num_agents` ≤ 10 and $1 \leq$ `num_rounds` ≤ 5, so that any concrete workflow has a bounded LLM-call budget regardless of the optimizer's choices. A *round* is one complete pass of the operator's communication pattern (every worker acts once); operators with `num_rounds` > 1 repeat this pass, each time conditioned on the state produced by the previous pass. Table VIII summarizes the parameters and call counts of each operator.

a) Sequential: A pipeline of single-agent calls. The agent is invoked once per prompt in `prompts`, with each call receiving the previous call's output as additional context, and the chain is repeated `num_rounds` times. There is no multi-agent coordination: only the worker role is consumed, `num_calls` = `len(prompts)` · `num_rounds`. This is the closest the operator set comes to a single-agent baseline.

b) Star: Parallel fan-out followed by a single aggregation step. `num_agents` workers run in parallel on the same problem, each with its own prompt; an aggregator then reads the concatenated worker outputs and produces a single answer. Workers are fully isolated and never see one another's responses. `num_calls` = `num_agents` + 1, mixing the worker and aggregator roles.

c) SharedMemory: Sequential writers to a shared document, with a final aggregation pass. On each of `num_rounds` rounds, the `num_agents` workers take turns reading the current document and appending a contribution; the aggregator then produces an answer from the final document. Communication is indirect through the document state. `num_calls` = `num_agents` · `num_rounds` + 1.

d) Debate: All-to-all explicit communication over `n_rounds` rounds. Each round, every agent revises its proposal having seen all other agents' current proposals; a final aggregator call collapses the last round's proposals into a single answer. Communication is direct, with full proposal visibility at every step. `num_calls` = `num_agents` · `n_rounds` + 1.

e) StarWithCoordinator: An iterative hub-and-spoke: the coordinator participates actively across `n_rounds` rounds. Each round, the coordinator broadcasts a directive, the `num_agents` workers respond in parallel, and the coordinator reads all responses to issue a revised directive; the directive after the final round is the operator's output. There is no worker-to-worker communication; the aggregator role is repurposed as the coordinator. `num_calls` = (`num_agents` + 1) · `n_rounds`.

f) DebateWithModerator: A moderated variant of DEBATE: a moderator agent injects a summary or steering prompt after every all-to-all exchange. Each agent now sees both its peers' proposals and the moderator's most recent intervention before producing its next response, and the moderator's output on the final round is the operator's answer. The aggregator role is repurposed as the moderator. `num_calls` = (`num_agents` + 1) · `n_rounds`, but with a richer signal at each step than STARWITHCOORDINATOR.

B. Parent Selection Policy

Parent workflow selection at each AFlow expansion step follows the mixed policy of Zhang et al. [33]. Let $s_1, \ldots, s_n$

TABLE VII

PRINCIPLE 3 (RQ3): LONGER, STRUCTURALLY RICH PROMPTS RAISE SCORE. PER-PUZZLE EFFECT SIZE Δ SCORE (HIGH ARM − LOW ARM) ON THE TRAIN AND HELD-OUT TEST SPLITS. FEATURE DEFINITIONS: **MEAN CHARS** = MEAN CHARACTER LENGTH OF THE MODULE-LEVEL STRING-ASSIGNMENT PROMPT TEMPLATES IN `prompt.py`, MEDIAN-SPLIT WITHIN EACH PUZZLE (HIGH ARM = ABOVE THE PUZZLE'S MEDIAN); **#PLACEH.** = TOTAL OCCURRENCE COUNT OF {name}-STYLE FORMAT PLACEHOLDERS (MATCHED BY `r"\{[a-zA-Z_][a-zA-Z0-9_]*\}"`, WITH REPEATS) ACROSS ALL MODULE-LEVEL PROMPT-TEMPLATE STRINGS IN `prompt.py`, MEDIAN-SPLIT WITHIN EACH PUZZLE. k_+/n AND k_-/n COUNT CELLS WHOSE 95% CLUSTER-WALD CI IS STRICTLY ABOVE / BELOW ZERO (OUT OF TESTABLE CELLS). HEADLINE: MEAN CHARS +11/13 ON TRAIN (MEDIAN Δ = +0.073) AND +6/13 ON TEST (MEDIAN Δ = +0.118); #PLACEH. +9/13 ON TRAIN (MEDIAN Δ = +0.102) AND +7/13 ON TEST (MEDIAN Δ = +0.059).

	mean chars		#placeh.	
Puzzle	tr	te	tr	te
Campsite	+0.094	+0.123	+0.132	+0.145
CarPainting	+0.006	+0.015	+0.008	+0.049
CryptoKKA	+0.171	+0.132	+0.154	+0.132
CryptoKPA	+0.052	+0.028	+0.056	+0.029
Game24	+0.047	+0.025	+0.048	+0.020
HamiltonianPath	+0.052	+0.069	+0.011	−0.014
Kakurasu	+0.256	+0.119	+0.270	+0.076
MagicSquare	+0.069	+0.042	+0.119	+0.061
StackPermutation	+0.074	+0.130	+0.102	+0.059
Sudoku	+0.069	+0.189	+0.184	+0.200
Sudoku2	+0.102	+0.187	+0.135	+0.135
SumSkyscraper	+0.073	+0.016	+0.025	+0.035
ZebraLogic	+0.122	+0.118	+0.072	−0.010
k_+/n	11/13	6/13	9/13	7/13
k_-/n	0/13	0/13	0/13	0/13
median Δ	+0.073	+0.118	+0.102	+0.059

Operator	Communication	Structure	Params	Calls
SEQUENTIAL	single agent	linear pipeline	$r, \mathbf{p}$	$\lvert\mathbf{p}\rvert \cdot r$
STAR	isolated workers	fan-out → agg	$n, \mathbf{p}_w, p_a$	$n+1$
SHAREDMEMORY	shared doc	writes → agg	$n, r, \mathbf{p}_w, p_a$	$n \cdot r + 1$
DEBATE	all-to-all	exchange → agg	$n, r, \mathbf{p}_w, p_a$	$n \cdot r + 1$
STARW/COORDINATOR	coord ↔ workers	iterative hub-spoke	$n, r, \mathbf{p}_w, p_a$	$(n+1) \cdot r$
DEBATEW/MODERATOR	all-to-all + mod	debate + mod/round	$n, r, \mathbf{p}_w, p_a$	$(n+1) \cdot r$

TABLE VIII

COMPOSABLE OPERATOR PRIMITIVES. $n \in [1, 10]$ WORKERS; $r \in [1, 5]$ ROUNDS. $\mathbf{p}_w$: LIST OF WORKER PROMPTS (ONE PER AGENT); p_a: SECOND-ROLE PROMPT (AGGREGATOR / COORDINATOR / MODERATOR). SEQUENTIAL IS SINGLE-AGENT AND TAKES A SINGLE PROMPT LIST $\mathbf{p}$. SEE FIG. 2 FOR THE TOPOLOGY OF EACH OPERATOR AND FIG. 3 FOR HOW OPERATORS COMPOSE.

denote the scores of the top-k previously explored candidates ($k = 4$), with $s_{\max} = \max_i s_i$. The probability of selecting candidate i is

$$P_{\text{mixed}}(i) = \lambda \cdot \frac{1}{n} + (1-\lambda) \cdot \frac{\exp\big(\alpha \cdot (s_i - s_{\max})\big)}{\sum_{j=1}^{n} \exp\big(\alpha \cdot (s_j - s_{\max})\big)}, \quad (4)$$

where $\lambda = 0.2$ controls the trade-off between uniform sampling and score-weighted sampling, and $\alpha = 0.4$ controls the temperature of the score-weighted component. The shift by $s_{\max}$ stabilizes the softmax against numerical overflow without altering the resulting distribution. The uniform component prevents collapse onto the optimizer's prior over high-frequency operators (cf. Table XI) and ensures broad trajectory coverage across rounds (§ A).

C. AFlow Integration

The AFlow codebase [33] was modified along three axes. First, the operator registry was replaced with the six primitives of §C-A, with corresponding updates to the optimizer-facing operator manual injected into the modification prompt (§D-D). Second, the dataset adapter was extended to interface with the Enigmata generators and verifiers; this required matching Enigmata's per-puzzle scoring contracts to AFlow's evaluator interface and routing the difficulty-bundle parameter through the existing task-config plumbing. Third, the per-round telemetry was extended to record 1) per-call worker/aggregator role attribution, and 2) the prompts emitted by the optimizer for each instantiation.

D. Enigmata Specification

Enigmata [3] bundles 36 abstract-reasoning puzzles, each paired with a parametric generator and a deterministic, rule-based verifier. The verifier performs an exact match against canonicalized ground-truth output and admits no partial credit, with the per-instance score $\in \{0, 1\}$. We restrict our experiments to the 30 puzzles for which both generator and verifier expose programmatic interfaces; the remaining six puzzles are excluded. Difficulty bundles are inherited from the Enigmata partition, which was calibrated against GPT-4o pass rates [3]: 27 of the 30 retained puzzles expose the full {Easy, Medium, Hard} bundle, while three (Hitori, LightUp, and TicTacToe) expose only {Medium, Hard}.

For each (puzzle, bundle) cell, we sample $M = 20$ instances at search initialization and reuse this fixed set across all $K = 5$ AFlow searches and all workflow rounds within a search. Reusing instances within a cell ensures that score variation across workflows reflects workflow structure rather than instance resampling. Mixed-difficulty bundles are left to future work. The category breakdown of the different puzzles is reported in Table XIV.

E. Filtered Puzzles and Failure Modes

Of the 30 retained puzzles, four (EightPuzzle, NinePuzzle, FifteenPuzzle, and SixteenPuzzle), all in the Sequential category, were excluded from the analysis corpus. On these puzzles, the maximum score across the entire set was 0: no workflow ever produced a verifier-accepted solution at any difficulty level. Inspection of representative completions confirmed that the failure mode is uniform across operators and difficulty bundles, with the worker model either emitting an incorrect move sequence or asserting that no solution exists. Because admission to the predictability gate requires non-degenerate score variance within a puzzle, these four puzzles cannot contribute to any per-puzzle effect-size estimate; retaining them would inflate n_{total} without adding statistical power. The filtered analysis corpus therefore comprises 26 puzzles and 28,568 workflows (Table IX).

A secondary failure mode, applying to the executable-workflow filter rather than to whole puzzles, removed 1,031 workflows (3.0% of the post-bundle corpus) for which the AFlow Optimizer produced a graph that could not be evaluated,

typically because a custom prompt referenced an undefined placeholder or a deleted prompt key. These failures are uniformly distributed across the (puzzle, bundle, model) grid and do not bias the structural feature distribution conditional on successful execution.

Appendix D
Analysis Phase Methodology

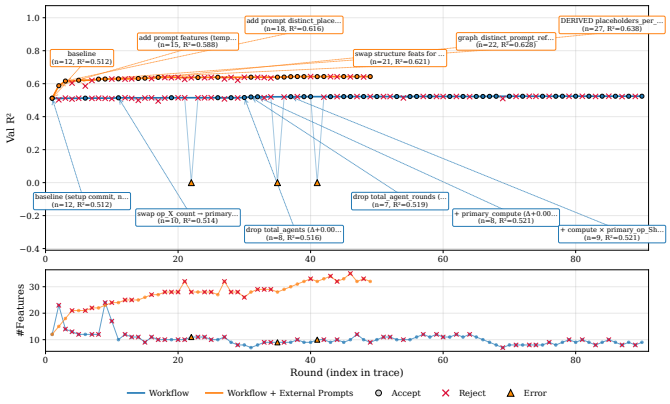


Fig. 7. Example of two AutoResearch searches, one exploring with access to only the executable workflow code without custom prompts, the other with access to both. The number of rounds differs based on when the AutoResearch instance considers it is stalling.

A. Feature Engineering and Models

The feature set $\boldsymbol{a} = \phi(W)$ is computed only from the workflow's own source – the operator graph and the prompts – plus the two allow-listed problem-side descriptors (puzzle family and difficulty bundle). The shared operator library, while readable as a reference, is identical across every workflow and therefore cannot produce a feature. The base parsers cover counts and structural summaries of operator calls, prompt-length statistics, placeholder topology, and a handful of style markers (step-by-step, format blocks); the autonomous loop iterates on them.

a) Train / val / test split provenance: The three-way split is assigned at the level of the AFlow *search*, not the workflow. Each search is identified by a stable hash of its full configuration; the assignment is deterministic per hash with a fixed seed (so a re-run on the same configuration lands in the same split), and the corpus-flattening step only inherits this assignment without resampling. After flattening, every workflow lives at a content-free identifier `workflow_id` = sha1(`search_hash` : `round`)[: 16]; the legacy `(task, round, search_hash)` path components are removed from the directory structure, and the two task-/round-dependent imports inside each workflow's graph source are rewritten to neutral, content-free forms. Together, these guarantee that the workflow source is independent of its on-disk path. The reproducibility contract on the loop is stronger than just split determinism: on any commit, re-running the analysis script after the flattening step must reproduce the committed findings snapshot and the structured metrics record byte-for-byte. This is enforced through pinned random seeds inside every random-state-bearing step of the regression pipeline, a frozen dependency environment, and a prohibition on machine-local state beyond the workflow corpus and a single targets cache.

b) Identifier exclusions: The row-level identifiers (`workflow_id`, round index, configuration hash) are forbidden as features, including in any reconstructed or hashed form. The split column itself is also forbidden.

c) Target-derived feature exclusions: No feature may depend on the primary target m or on the workflow's total cost on any row. This prohibition extends to target encoding of categoricals (smoothed or otherwise), per-group target aggregates, residuals, ranks, and label-derived difficulty priors. A label-derived feature is, by construction, never a property of the workflow's design.

B. AutoResearch Details

Every iteration of the autonomous loop atomically commits, alongside the analysis script, a fixed set of artifacts that together render any past iteration replayable from the git history alone:

- `metrics.json`: structured metrics, feature names, and top coefficients (the time-series source for Fig. 7);
- `feature_names.json`: the active feature list as a flat array;
- `train.py`: the training script driving feature engineering and surrogate fitting.

The fitted model and preprocessor binaries are written outside the git tree and overwritten on each run; checking out a past commit and re-running the analysis script regenerates them, and serves as the byte-for-byte reproducibility check.

a) Per-commit log: A per-commit log table accompanies the artifacts and functions as the running lab notebook of the autonomous loop. Every attempt, whether accepted, discarded, or aborted by a runtime error, appears in this log because the branch is append-only by construction. The verdict labels are the agent's own classification, recorded at the time of commit; they are included as a reproducibility artifact rather than a primary result, and are not used as features by the surrogate.

C. Caveats and Mitigated Failure Modes

The combination of an LLM-driven feature proposer and a sandboxed evaluation loop is, by construction, susceptible to feature constructions that exploit incidental dataset structure rather than workflow design. Two failure modes were observed during early AutoResearch runs and addressed before the design principles reported were finalized.

a) Round index as a feature: An early proposal used the AFlow round index as a direct numerical feature, exploiting the fact that later-round workflows tend to be selected from higher-scoring parents and therefore inherit a positive score correlation that is unrelated to architectural design. The mitigation strips the round index from the feature-side metadata exposed to the feature-mining loop and restricts the available artifacts to the Python implementations of the workflows. The round index is retained in evaluation metadata for analysis bookkeeping but is never visible to ϕ.

b) Train-mean leakage: A second early proposal computed the mean training-set score per (puzzle, bundle) cell and added it as a feature on the validation and test rows, yielding spuriously high R^2. The mitigation extends the sandbox prompt with an explicit prohibition on aggregate statistics derived from the target column, regardless of split.

D. Prompts

We provide LLM-MAS adapted parts of the prompts used by AFlow to conduct the workflow searches, tailored to our operator set.

AFlow MAS Optimize Prompt

```
You are building a Graph and corresponding Prompt to jointly solve
{type} problems using multi-agent system (MAS) operators.
Referring to the given graph and prompt, which forms a basic example
of a {type} solution approach,
please reconstruct and optimize them.
You can add, modify, or delete nodes, parameters, or prompts.
Include your
single modification in XML tags in your reply.
Ensure they are complete and correct to avoid runtime failures.
The available operators are multi-agent topologies:
Sequential, Star, SharedMemory, Debate, StarWithCoordinator,
DebateWithModerator.
Each operator takes self.llm (worker) and self.aggregator_llm
(aggregator/coordinator/moderator) as LLMs
— do NOT create or reference other LLMs.
You can combine operators, chain them, or adjust their parameters
(num_agents, n_rounds, prompts) to improve performance.
The graph complexity should not exceed 10 operator invocations.
Use logical and control flow (IF-ELSE, loops) for enhanced graph design.

Ensure that all prompts required by the current graph from prompt_custom
are included.
Exclude any unused prompts.
Output the modified graph and all necessary prompts in
prompt_custom (if needed).
The generated prompt must not contain any placeholders other than the
ones supported by the operators.
Considering information loss, complex graphs may yield better results,
but insufficient information transmission can omit the solution.
It's crucial to include necessary context during the process.
```

AFlow MAS Component Usage

Here are examples of using the MAS operators in the graph:

```
# Sequential: linear pipeline, each step gets the previous output
seq = operator.Sequential(llm=self.llm, num_rounds=2,
    prompts=["Solve: {problem}",
    "Review and improve: {previous_output}"])
result = await seq(problem)

# Star: parallel workers then aggregation
star = operator.Star(worker_llm=self.llm,
aggregator_llm=self.aggregator_ll
, num_agents=3,
    worker_prompts=["Solve this: {problem}"] * 3,
    aggregator_prompt="Given these solutions: {worker_outputs}
    Pick the best for: {problem}")
result = await star(problem)

# Debate: agents debate then aggregation
debate = operator.Debate(worker_llm=self.llm,
    aggregator_llm=self.aggregator_llm,
    num_agents=3,
    worker_prompts=["Solve: {problem} Other proposals:
    {all_proposals}"] * 3,
    aggregator_prompt="Final answer for {problem} given:
    {all_proposals}",
    n_rounds=2)
result = await debate(problem)

# All MAS operators return dict with key 'response'
answer = result['response']
```

`

```
The available LLMs are self.llm (worker) and self.aggregator_llm
(aggregator/coordinator/moderator).

Supported placeholders per operator:
- Sequential: {problem}, {previous_output}
- Star: worker_prompts use {problem}; aggregator_prompt uses {problem},
{worker_outputs}
- SharedMemory: worker_prompts use {problem}, {document}, {agent_id},
    {round};
    aggregator_prompt uses {problem}, {document}
- Debate: worker_prompts use {problem}, {all_proposals}, {round},
    {agent_id}; aggregator_prompt uses {problem}, {all_proposals}
- StarWithCoordinator: worker_prompts use {problem}, {directive},
    {round}, {agent_id}; coordinator_prompt uses {problem},
    {worker_outputs}, {round}, {is_final}
- DebateWithModerator: agent_prompts use {problem}, {all_proposals},
    {moderator_input}, {round}, {agent_id};
    moderator_prompt uses {problem}, {all_proposals}, {round},
    {is_final}

You can also define custom prompts in prompt_custom and reference them.

**Combining multiple operators or chaining them can enhance performance.
Try different topologies.**
```

Appendix E
Corpus Details and Additional Analyses

A. Statistics

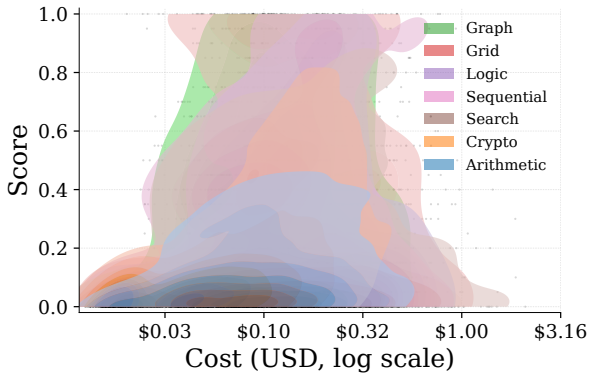


Fig. 8. **Design-Space Coverage.** Joint distribution of workflow evaluation accuracy versus inference cost (USD, log scale) across task categories, spanning a broad spectrum of accuracy-cost trade-offs.

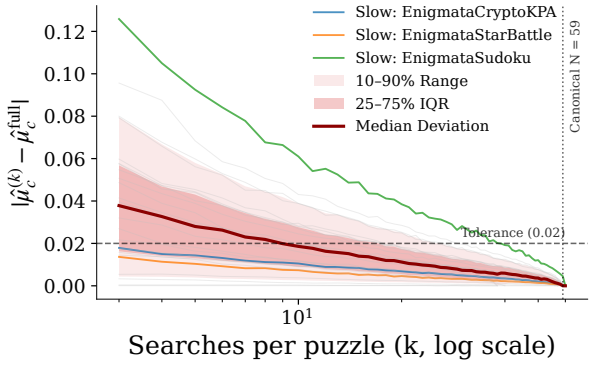


Fig. 9. **Sampling Sufficiency.** Absolute deviation of the running cell mean from its full-sweep value across 26 puzzles as a function of search count k. Rapid convergence below 0.02 confirms estimator stability.

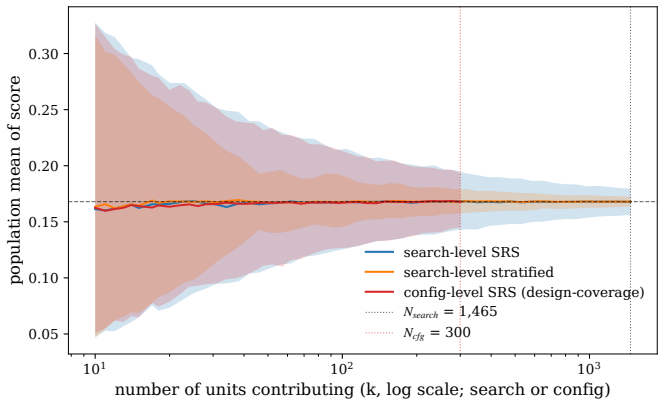


Fig. 10. Convergence of the population-mean score estimator. For each k (log scale), we draw B=2,000 bootstrap replicates of k resampling units with replacement and plot the bootstrap median (line) and central 95% band of $\hat{\mu}(k) = \text{mean}_{\text{rows}}(\text{score})$. Three resampling units: SRS over searches (blue), SRS stratified by (dataset, difficulty) (orange), SRS over configurations (red). Dashed horizontal line: full-population mean. Dotted verticals: N_{search} (black), N_{cfg} (red). Cleaned subset (4 sliding-tile puzzles dropped): 1,465 searches, 300 configurations, 26 puzzles, $\hat{\mu}_{\text{full}}$=0.168.

Split	Count	Percentage
Total	28,568	100.0%
Train	19,822	69.4%
Validation	4,278	15.0%
Test	4,468	15.6%

TABLE IX
EFFECTIVE WORKFLOW COUNTS ACROSS SPLITS. OF 1,701 IN-SCOPE COMPLETED AFLOW SEARCHES, EACH GENERATING UP TO 20 WORKFLOWS (34,020 CANDIDATES), 32,989 ARE EXECUTABLE. A FURTHER DROP TO 28,568 WORKFLOWS REMOVES THE FOUR FILTERED SLIDING-TILE PUZZLES (EIGHT/FIFTEEN/NINE/SIXTEENPUZZLE), WHERE NO WORKFLOW EVER SCORED ABOVE ZERO. THESE ARE THEN DIVIDED INTO THE TRAIN/VAL/TEST SPLIT.

B. Cluster-Robust Inference and Small-Cluster Coverage

The per-cell confidence intervals reported in §V-B and in Tables V–VII are computed using cluster-Wald standard errors with the AFlow search identifier as the clustering unit. Clustering at the search level rather than at the workflow level is the appropriate choice: workflows generated within the same MCTS trace are not independent draws from the workflow population, since the optimizer conditions each expansion on the score history of its parent. The cluster-Wald estimator is consistent in the large-cluster-count limit, with standard recommendations requiring at least $G \geq 15$ clusters per arm for nominal coverage.

a) Train-split cluster support: On the training split, per-arm cluster counts across the cells in Tables V–VII range from 19 to 47. A single contrast (TicTacToe × `op_SharedMemory_count`) sits at the lower bound with $G = 19$ clusters in its smaller arm; all other train-side contrasts have $G \geq 24$ per arm. The train-split CIs therefore lie within the regime in which cluster-Wald inference is well-supported, and the train-side strict-sign verdicts (k_+/k_-) are unaffected by the issue described below.

b) Test-split cluster support: The held-out test split is, by construction, more sparsely supported at the cluster level: the search-level split assigns each AFlow search to exactly one of train, validation, and test, and the test split contains ∼ 15% of searches. Across the cells reported in the test columns of Tables V–VII, per-arm cluster counts range from 4 to 14, with the majority of cells below $G = 10$. The least-supported cells, both with $G = 4$ per arm, would fall outside the asymptotic regime.

Task	Easy ($)	Medium ($)	Hard ($)	Mean ($)
StarBattle	$0.0110	$0.0096	$0.0099	$0.0102
SumSkyscraper	$0.0108	$0.0092	$0.0102	$0.0100
Sudoku	$0.0096	$0.0105	$0.0090	$0.0097
ZebraLogic	$0.0062	$0.0097	$0.0116	$0.0091
Campsite	$0.0065	$0.0104	$0.0102	$0.0091
Kakurasu	$0.0094	$0.0063	$0.0113	$0.0090
Hitori	–	$0.0093	$0.0086	$0.0090
HamiltonianCycle	$0.0083	$0.0073	$0.0087	$0.0081
LightUp	–	$0.0070	$0.0082	$0.0076
CarPainting	$0.0079	$0.0070	$0.0075	$0.0075
Skyscraper	$0.0067	$0.0074	$0.0076	$0.0072
HamiltonianPath	$0.0058	$0.0072	$0.0085	$0.0072
Slant	$0.0069	$0.0071	$0.0065	$0.0068
Binario	$0.0062	$0.0069	$0.0064	$0.0065
MagicSquare	$0.0054	$0.0059	$0.0077	$0.0063
StackPermutation	$0.0054	$0.0067	$0.0063	$0.0061
Minesweeper	$0.0059	$0.0058	$0.0067	$0.0061
Maze	$0.0064	$0.0058	$0.0049	$0.0057
Twiddle	$0.0040	$0.0072	$0.0056	$0.0056
Countdown	$0.0057	$0.0061	$0.0047	$0.0055
SixteenPuzzle	$0.0047	$0.0043	$0.0049	$0.0046
FifteenPuzzle	$0.0044	$0.0044	$0.0045	$0.0044
Sudoku2	$0.0045	$0.0044	$0.0044	$0.0044
NaturalLanguageNavigation	$0.0030	$0.0041	$0.0060	$0.0043
Game24	$0.0041	$0.0045	$0.0043	$0.0043
EightPuzzle	$0.0042	$0.0039	$0.0043	$0.0041
NinePuzzle	$0.0040	$0.0041	$0.0037	$0.0040
TicTacToe	–	$0.0037	$0.0041	$0.0039
CryptoKKA	$0.0040	$0.0029	$0.0030	$0.0033
CryptoKPA	$0.0037	$0.0020	$0.0031	$0.0030
Avg.	$0.0061	$0.0064	$0.0067	$0.0064

TABLE X
AVERAGE COST IN USD TO EVALUATE ONE WORKFLOW AGAINST A SINGLE TARGET PUZZLE INSTANCE, BY TASK AND DIFFICULTY. "–" MARKS (PUZZLE, DIFFICULTY) CELLS THAT ENIGMATA'S GENERATOR DOES NOT PROVIDE.

Operator	# Workflows	Percentage	Score		Cost	
			w	w/o	w	w/o
Sequential	13,473	47.16	0.14	0.19	0.14	0.15
Star	12,716	44.51	0.16	0.17	0.13	0.16
Debate	6,373	22.31	0.18	0.17	0.13	0.15
SharedMemory	5,948	20.82	0.23	0.15	0.25	0.12
DebateWithModerator	3,382	11.84	0.19	0.16	0.19	0.14
StarWithCoordinator	2,502	8.76	0.16	0.17	0.24	0.14

TABLE XI
OPERATOR PRESENCE OVER THE FILTERED ANALYSIS CORPUS (28,568 WORKFLOWS). FOR EACH OPERATOR WE CONTRAST THE MEAN OF *SCORE* AND *COST* BETWEEN WORKFLOWS THAT CONTAIN THE OPERATOR (*w*) AND THOSE THAT DON'T (*w/o*).

C. Candidate Hypothesis Set

We explored various styles of prompts and constraints for the AutoResearch loop to mitigate overfitting, leading to different

Operator combination	# WFs	Percentage	Score		Cost	
			w	w/o	w	w/o
Star → Seq	4,564	15.98	0.14	0.17	0.12	0.15
Star	4,125	14.44	0.15	0.17	0.07	0.16
Debate	2,346	8.21	0.17	0.17	0.09	0.15
SharedMemory → Seq	2,158	7.55	0.18	0.17	0.23	0.14
Seq	2,078	7.27	0.09	0.17	0.05	0.16
Debate → Seq	2,041	7.14	0.14	0.17	0.12	0.15
SharedMemory	1,748	6.12	0.25	0.16	0.20	0.14
DebateWithModerator	1,510	5.29	0.19	0.17	0.12	0.15
StarWithCoordinator	1,085	3.80	0.15	0.17	0.21	0.15
StarWithCoordinator → Seq	741	2.59	0.13	0.17	0.24	0.14

TABLE XII
TOP-10 MOST-USED OPERATOR COMBINATIONS OVER THE FILTERED ANALYSIS CORPUS (28,568 WORKFLOWS), RANKED BY FREQUENCY. OPERATORS IN A SEQUENCE ARE CONNECTED BY →. FOR EACH ROW, WE CONTRAST THE MEAN OF *SCORE* AND *COST* BETWEEN WORKFLOWS THAT USE THE EXACT SEQUENCE (*W*) AND THE REST OF THE CORPUS (*W/O*).

exploration trajectories and saving protocols; we narrowed our focus to 300 feature-engineering trials spread across 10 main AutoResearch searches. We used features from these different search traces to generate 70 candidate binarized feature contrasts (spanning the three granularities in Table XV), covering hypotheses related to prompt information, number of rounds and agents in workflows, operator positions, and derived features combining merging features.

For instance the two search traces from Fig. 7 cover two setups, one with access to both `graph.py` (containing the executable workflow code) and `prompt.py` (custom generated prompts), and the other with access only to `graph.py`. The search with access to prompt data ran for 49 trials, 46.9% of which were accepted and grew the feature set from 12 to 32. The search with access only to `graph.py` started with an initial set of 12 features and decreased it to 9 features over 90 trials. Not all trials necessarily optimize features, as the automated search sometimes attempts to optimize model hyperparameters or change the model class directly.

APPENDIX F
SINGLE-BACKBONE BASELINE PERFORMANCE

To contextualize the workflow-level scores, Table XIV reports the one-shot pass rate of the three model backbones (GPT-5.4, GPT-5.4-mini, GPT-5.4-nano) used as the worker and aggregator policies in our experiments, evaluated directly on the per-puzzle bundle without any multi-agent scaffolding. The same table reports the workflow-level mean score $\bar{\mu}_W$, the workflow-level maximum score $\bar{\mu}_W^{\max}$, and the cost in USD of the maximum-scoring workflow.

Two patterns are noteworthy. First, on puzzles where workflows substantially outperform the strongest single-backbone baseline, for example, NaturalLanguageNavigation (0.10 baseline, 0.89 workflow mean) and Sudoku2 (0.98 baseline, 0.95 workflow mean), the workflow-level maximum approaches or matches the empirical ceiling, suggesting that the AFlow search saturates these puzzles. Second, on puzzles where all three baselines score zero (Slant, LightUp, Minesweeper, EightPuzzle, NinePuzzle, FifteenPuzzle, SixteenPuzzle), workflows reach at most 0.05 on Slant and LightUp, while the four sliding-tile puzzles remain at 0.00 throughout and are excluded from the analysis corpus.

APPENDIX G
COMPUTATIONAL COST ANALYSIS

Table X reports the average cost in USD to evaluate one workflow against a single puzzle instance, broken down by puzzle and difficulty. Difficulty has a small but consistent effect on cost: the cross-puzzle mean rises from \$0.0061 (Easy) to \$0.0067 (Hard). The 20-instance validation bundle (one round), therefore, costs 20× the per-instance figure. Assuming the average cost per instance, the evaluation of the workflows from one AFlow search is in the order $\$0.0064 \times 20 \times 20$, as such for $1,740$ searches, one can expect $\sim \$4,454$, to search or evaluate the workflows. The workflow evaluation does not call the optimizer LLM, which incurs an additional cost for the search.

APPENDIX H
TRACE EXAMPLES

The search trace shown in Fig. 11 corresponds to a single search on a NaturalLanguageNavigation bundle puzzle on which workflow-level performance substantially exceeds the single-backbone baseline (Table XIV: baseline ≤ 0.15, workflow mean 0.89).

a) Per-round and best-so-far divergence: The per-round score and the best-so-far envelope diverge substantially throughout the search: the running maximum reaches 0.90 at round 16, yet four of the final five rounds score at or below 0.75. This off-envelope variance is one motivation for our full-trace sampling protocol: retaining all $N = 20$ workflows from each search rather than only the argmax preserves the design-space signal carried by the intermediate and post-peak rounds, which an argmax-only corpus would discard.

b) Operator-composition trajectory: The bottom-left panel shows a transition from single-operator workflows in rounds 1–6 to two-operator compositions thereafter. During the single-operator phase, the optimizer cycles through SEQUENTIAL, STAR, and DEBATE as the base topology; from round 7 onward, the search settles into two-operator chains and explores variants within this richer topology class for the remainder of the trace.

c) MCTS lineage and within-search dependence: The right panel makes the dependence structure of the search explicit. The gold path links the root to several intermediate improvements to the peak at r_{16}, with each step on the path corresponding to a strict score improvement (green edges); off-path explorations such as r_4, r_5, r_{12}, and r_{17} either regress relative to their parent or fail to advance the running maximum.

APPENDIX I
GATE CALIBRATION

The validation thresholds $R^2 > 0.05$ and Spearman $\rho > 0.30$ used in the predictability gate (§ V-B) were chosen to lie a non-trivial margin above the performance attainable by chance

Puzzle	# WFs	Mean	Std	Min	25%	50%	75%	Max	Skew	Kurt.
Graph										
HamilCycle	1,164	0.073	0.101	0.00	0.00	0.00	0.10	0.40	1.40	1.10
HamilPath	1,147	0.150	0.158	0.00	0.00	0.10	0.25	0.70	0.96	-0.05
Maze	1,075	0.479	0.221	0.00	0.35	0.50	0.65	1.00	-0.21	-0.29
NLNavigate	1,120	0.561	0.322	0.00	0.30	0.65	0.85	1.00	-0.36	-1.18
Logic										
ZebraLogic	1,159	0.184	0.266	0.00	0.00	0.05	0.30	1.00	1.42	0.90
Grid										
Binario	1,120	0.208	0.249	0.00	0.00	0.10	0.30	1.00	1.44	1.50
Campsite	1,136	0.088	0.167	0.00	0.00	0.00	0.10	0.85	2.12	3.81
MagicSquare	1,166	0.131	0.175	0.00	0.00	0.05	0.15	0.65	1.48	1.02
Skyscraper	1,182	0.003	0.014	0.00	0.00	0.00	0.00	0.15	4.76	26.62
StarBattle	1,197	0.024	0.067	0.00	0.00	0.00	0.00	0.50	3.59	13.74
Sudoku	1,199	0.300	0.355	0.00	0.00	0.10	0.55	1.00	0.83	-0.81
Sudoku2	1,178	0.698	0.374	0.00	0.55	0.90	1.00	1.00	-1.04	-0.52
SumSkyscrap	1,152	0.050	0.140	0.00	0.00	0.00	0.00	0.90	3.22	10.01
Crypto										
CryptoKKA	1,114	0.121	0.210	0.00	0.00	0.00	0.15	1.00	2.08	3.67
CryptoKPA	1,191	0.033	0.077	0.00	0.00	0.00	0.05	0.65	3.74	17.38
Arithmetic										
Countdown	1,137	0.038	0.089	0.00	0.00	0.00	0.05	0.70	3.66	15.95
Game24	1,144	0.107	0.122	0.00	0.00	0.05	0.15	0.70	1.32	1.17
Search										
Hitori	768	0.039	0.102	0.00	0.00	0.00	0.05	0.85	4.49	24.24
Kakurasu	1,159	0.136	0.240	0.00	0.00	0.00	0.15	0.95	1.81	2.13
LightUp	732	0.000	0.005	0.00	0.00	0.00	0.00	0.05	10.93	117.82
Minesweeper	1,079	0.006	0.038	0.00	0.00	0.00	0.00	0.60	10.32	129.82
Slant	1,136	0.000	0.002	0.00	0.00	0.00	0.00	0.05	23.80	565.49
TicTacToe	737	0.376	0.203	0.00	0.25	0.40	0.50	0.85	-0.27	-0.63
Sequential										
CarPainting	1,159	0.044	0.077	0.00	0.00	0.00	0.05	0.40	2.00	3.61
EightPuzzle	1,129	0.000	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00
FifteenPuzzle	1,152	0.000	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00
NinePuzzle	1,058	0.000	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00
SixteenPuzzle	1,082	0.000	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00
StackPerm	1,077	0.508	0.253	0.00	0.35	0.45	0.65	1.00	0.26	-0.43
Twiddle	1,140	0.014	0.052	0.00	0.00	0.00	0.00	0.45	4.61	23.52

TABLE XIII

PER-PUZZLE DISTRIBUTION OF score OVER THE FULL CORPUS OF 32,989 WORKFLOWS ON THE 30 ENIGMATA PUZZLES. ROWS ARE GROUPED BY CATEGORY (GRAPH → LOGIC → GRID → CRYPTO → ARITHMETIC → SEARCH → SEQUENTIAL), ALPHABETICAL WITHIN CATEGORY. THE FOUR SLIDING-TILE PUZZLES IN THE *SEQUENTIAL* CATEGORY (EIGHT/FIFTEEN/NINE/SIXTEENPUZZLE) HAVE ALL-ZERO DISTRIBUTIONS; THEY ARE EXCLUDED FROM THE ANALYSIS CORPUS USED BY THE OTHER TABLES.

under the same fitting procedure. To estimate this chance baseline, we construct a permutation null distribution: for each puzzle, the training-split score column was shuffled ten times, the LightGBM regressor was refit on each permuted training set, and the resulting model was evaluated on the unshuffled validation rows. This procedure preserves the marginal distribution of design features and the validation targets, while severing the feature–score association that the regressor is intended to capture; the resulting (R^2, ρ) pairs therefore characterize the signal that the fitting procedure attributes to noise alone.

The 95th percentile of the pooled null distribution, computed across all puzzles and across every feature-set variant considered during the AutoResearch iteration loop, satisfies $R^2_{p95} \leq -0.06$ and $\rho_{p95} \leq 0.23$ (visualized per puzzle in Fig. 12). The admission thresholds therefore exceed the worst-case null upper tail by at least 0.07 on ρ and at least 0.11 on R^2, so any admitted puzzle clears chance by a non-trivial and feature-set-invariant margin. The choice is conservative in the sense that it prioritizes false-negative robustness, admitting fewer puzzles in exchange for stronger guarantees on those that are admitted, which is appropriate for a downstream aggregation step in which a single floor or zero-variance puzzle would otherwise dilute the across-puzzle contrast.

APPENDIX J
BROADER SOCIETAL IMPACT

The contribution of this work is methodological rather than capability-oriented: our framework does not introduce a new agent or extend the capability frontier of any deployed system, but provides a methodology under which the determinants of agentic performance can be examined statistically. The design principles surfaced by our analysis, although not directly deployable, contribute incrementally to the general advance

Category	Puzzle	GPT-5.4	GPT-5.4-mini	GPT-5.4-nano	W_μ	W_{max}	USD($)@max
Arithmetic	Countdown	0.033	0.017	0.017	0.173	0.700	0.228
	Game24	0.150	0.083	0.050	0.288	0.700	0.220
Crypto	CryptoKKA	0.267	0.067	0.000	0.412	1.000	0.081
	CryptoKPA	0.050	0.050	0.000	0.139	0.650	0.238
Graph	HamilCycle	0.133	0.017	0.000	0.228	0.400	0.080
	HamilPath	0.300	0.100	0.017	0.363	0.700	0.233
	Maze	0.717	0.533	0.167	0.759	1.000	0.123
	NLNavigate	0.100	0.150	0.083	0.891	1.000	0.024
Grid	Binario	0.150	0.217	0.133	0.549	1.000	0.123
	Campsite	0.450	0.100	0.000	0.381	0.850	0.243
	MagicSquare	0.033	0.000	0.000	0.359	0.650	0.106
	Skyscraper	0.050	0.017	0.017	0.016	0.150	0.475
	StarBattle	0.033	0.000	0.000	0.121	0.500	0.501
	Sudoku	0.583	0.550	0.300	0.443	1.000	0.120
	Sudoku2	0.983	0.917	0.717	0.954	1.000	0.018
	SumSkyscrap	0.117	0.017	0.000	0.147	0.900	0.312
Logic	ZebraLogic	0.517	0.217	0.017	0.377	1.000	0.200
Search	Hitori	0.025	0.000	0.000	0.202	0.850	0.362
	Kakurasu	0.217	0.033	0.000	0.491	0.950	0.195
	LightUp	0.000	0.000	0.000	0.004	0.050	0.061
	Minesweeper	0.000	0.000	0.000	0.043	0.600	0.349
	Slant	0.000	0.000	0.000	0.002	0.050	0.315
	TicTacToe	0.675	0.125	0.225	0.641	0.850	0.066
Sequential	CarPainting	0.083	0.150	0.017	0.124	0.400	0.473
	EightPuzzle	0.000	0.000	0.000	0.000	0.000	0.014
	FifteenPuzzle	0.000	0.000	0.000	0.000	0.000	0.015
	NinePuzzle	0.000	0.000	0.000	0.000	0.000	0.013
	SixteenPuzzle	0.000	0.000	0.000	0.000	0.000	0.015
	StackPerm	0.467	0.433	0.150	0.849	1.000	0.062
	Twiddle	0.200	0.033	0.017	0.061	0.450	0.075

TABLE XIV

SINGLE-BACKBONE BASELINES: ONE-SHOT PASS RATE OF GPT-5.4, GPT-5.4-MINI, AND GPT-5.4-NANO EVALUATED DIRECTLY ON THE PER-PUZZLE BUNDLE WITHOUT SCAFFOLDING, ALONGSIDE WORKFLOW-LEVEL STATISTICS FROM OUR WORKFLOW CORPUS: MEAN SCORE W_μ, MAXIMUM SCORE $W_{\max}$, AND THE PER-INSTANCE INFERENCE COST OF THE MAXIMUM-SCORING WORKFLOW ($@ max). PUZZLES ARE GROUPED BY CATEGORY. THE FOUR ALL-ZERO SLIDING-TILE PUZZLES ARE LISTED FOR COMPLETENESS AND EXCLUDED FROM THE ANALYSIS CORPUS.

	Granularity	Prior Rule?	Representative Mined Principle
RQ1	*Entry-Point Topology*	✓	STAR as primary operator lowers score
RQ2	*Communication State*	✗	SHAREDMEMORY outperforms dialogue
RQ3	*Sub-Topological Prompts*	✗	Prompt specificity (length, `#placeh.`) helps

TABLE XV

THREE GRANULARITIES OF MINED DESIGN PRINCIPLES. WE EVALUATE OUR FRAMEWORK ACROSS PROGRESSIVELY FINER LEVELS OF WORKFLOW SPECIFICATION: FROM VERIFYING AN ESTABLISHED MACRO-TOPOLOGICAL FAILURE MODE (**RQ1**), TO COMPARING BLACKBOARD-STYLE SHARED STATE AGAINST CONVERSATIONAL TOPOLOGIES (**RQ2**), TO MINING SUB-TOPOLOGICAL CODE AND PROMPT TEMPLATE STRUCTURE (**RQ3**).

of agentic capability and inherit the dual-use risk common to capability research.

APPENDIX K
LLM USAGE

LLM usage is a central part of this work, as it relies on automated workflow generation, which includes code generation. Furthermore, the AutoResearch also conducts iterative data analysis.

We additionally use LLMs to polish the writing and check typos. LLMs were also used for programming aid, providing initial, skeleton implementations that were subsequently refined by the authors.

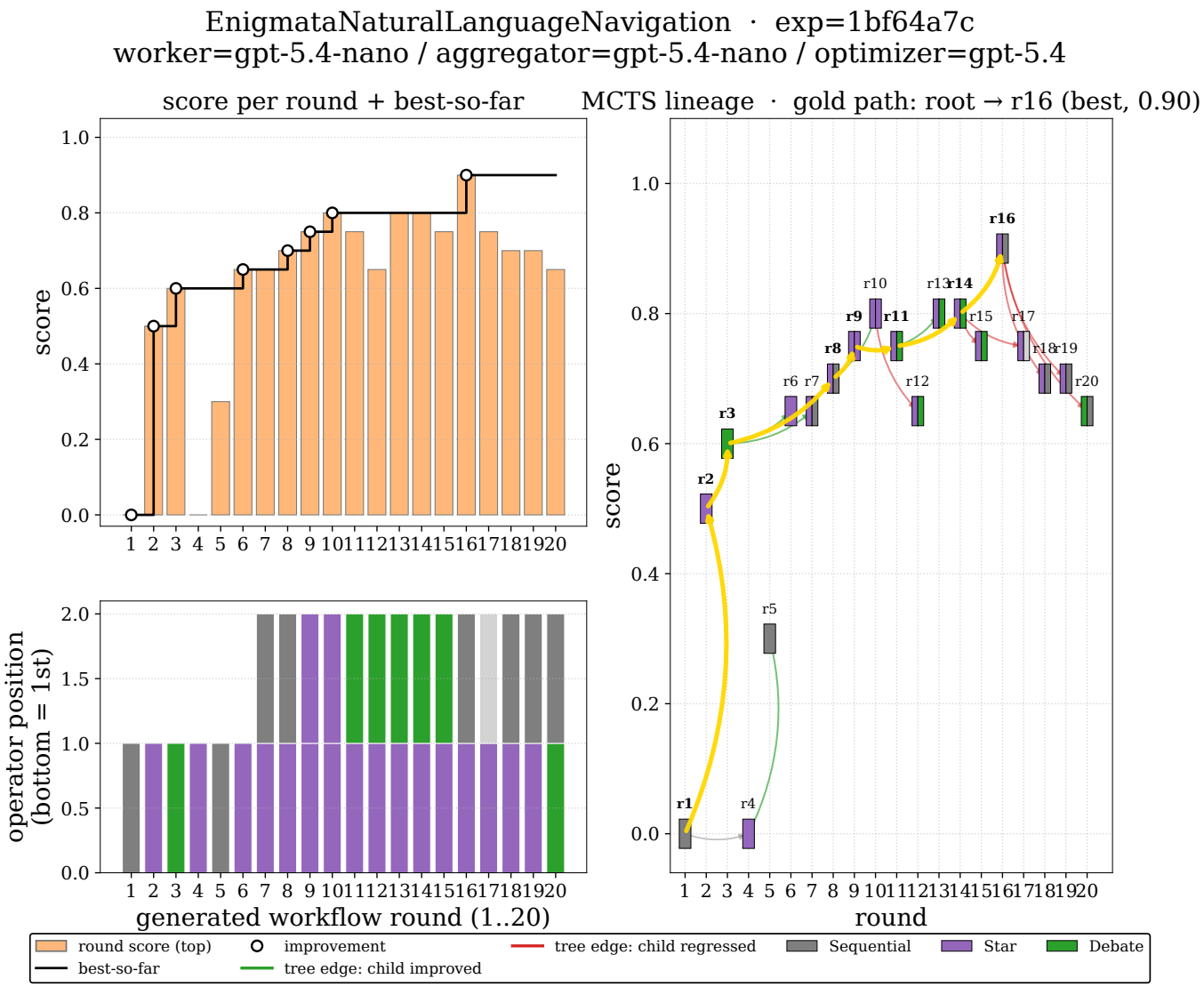


Fig. 11. AFlow search trace for a single (puzzle, model triple) cell on EnigmataNaturalLanguageNavigation, with worker and aggregator at `gpt-5.4-nano` and optimizer at `gpt-5.4` (`experiment_id = 1bf64a7c`). **Top-left:** per-round workflow score (orange bars) and best-so-far envelope (black step line); open circles mark rounds that improved the running maximum. **Bottom-left:** operator-composition trajectory, with bar height equal to the number of operator instantiations and color denoting operator type (gray = SEQUENTIAL, purple = STAR, green = DEBATE). **Right:** MCTS lineage tree, with each node labeled by round and positioned vertically by score. The gold path traces the root-to-best lineage (root → r_{16}, score 0.90); edge color encodes whether the child improved (green) or regressed (red) relative to its parent.

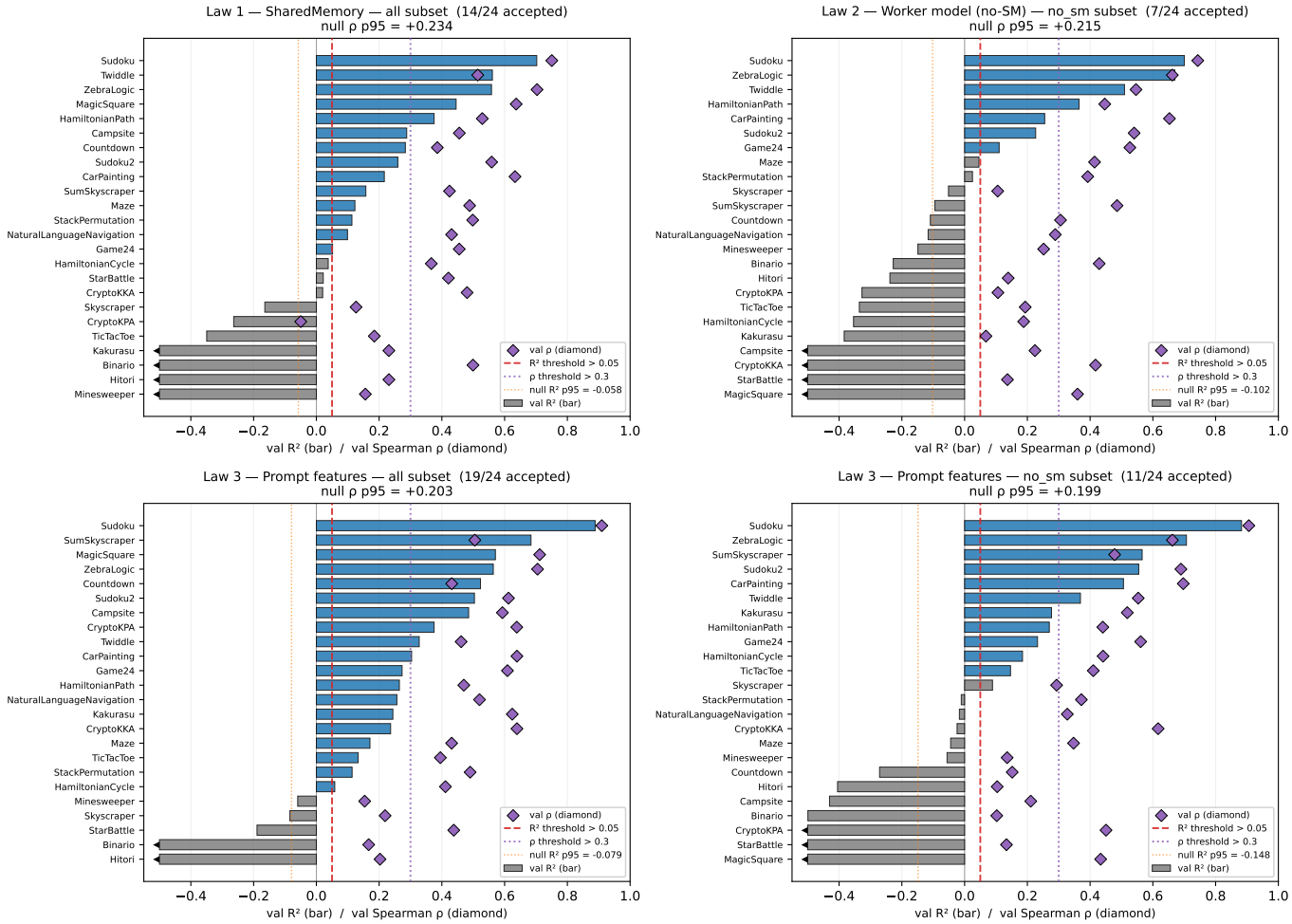


Fig. 12. **Per-puzzle predictability gate diagnostics across feature subsets.** For each puzzle, bars show validation R^2 and diamonds show validation Spearman ρ against the permutation-calibrated thresholds ($R^2 > 0.05$ red dashed line; $\rho > 0.30$ purple dotted line; 95th-percentile permutation null bounds shown in gray). A puzzle is admitted iff both metrics clear their respective thresholds on D_{val}.